

COMP 4423: FEATURE EXTRACTION

Easy Computer Vision

Xiaoyong Wei (魏驍勇)

x1wei@polyu.edu.hk



THE HONG KONG
POLYTECHNIC UNIVERSITY
香港理工大學

Opening Minds • Shaping the Future
啟迪思維 • 成就未來

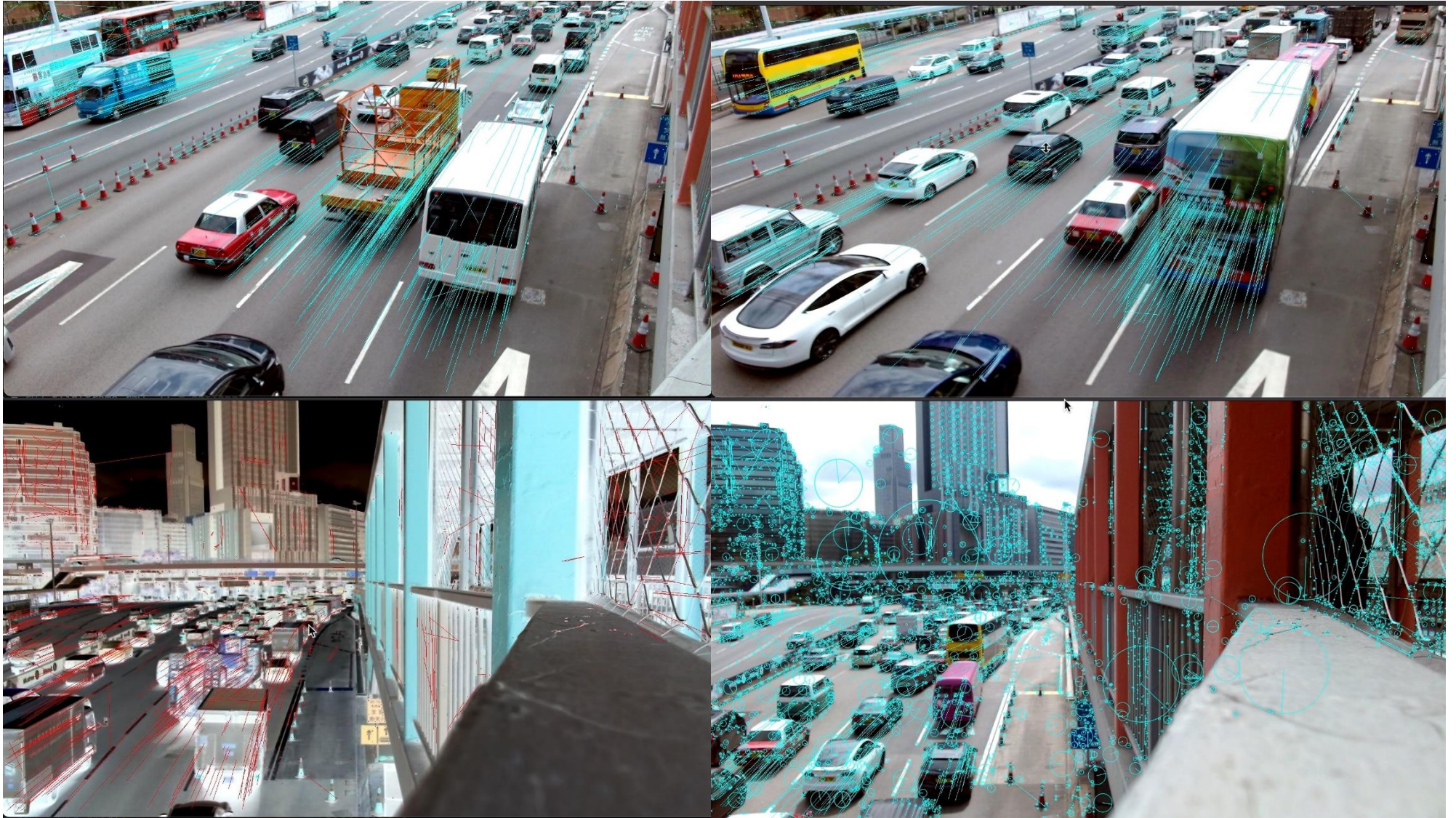


A New Toy





A New Toy





You will be able to implement it
after this lecture



Outline

- Features and Quantization
- Metrics (Distance and Similarity)
- Global and Local Features (Color Histograms, LBP, SIFT)



How do you group them?





It's easy!

	Yellow Hair, Blue Eyes, White Skin
	Black Hair, Brown Eyes, Yellow Skin
	Black Hair, Brown Eyes, Yellow Skin
	Yellow Hair, Brown Eyes, White Skin
	Black Hair, Brown Eyes, Black Skin



It's easy!

	Hair Color	Skin Color
	Yellow	White
	Black	Yellow
	Black	Yellow
	Yellow	White
	Black	Black



This is how human see it.

Is there a way for computers to “see” it?



Quantization

	Hair Color	Skin Color
	Yellow (2)	White (1)
	Black (3)	Yellow (2)
	Black (3)	Yellow (2)
	Yellow (2)	White (1)
	Black (3)	Black (3)



Great! Seems that now we can
build grouping rules easily
using if-else?



Right and wrong!

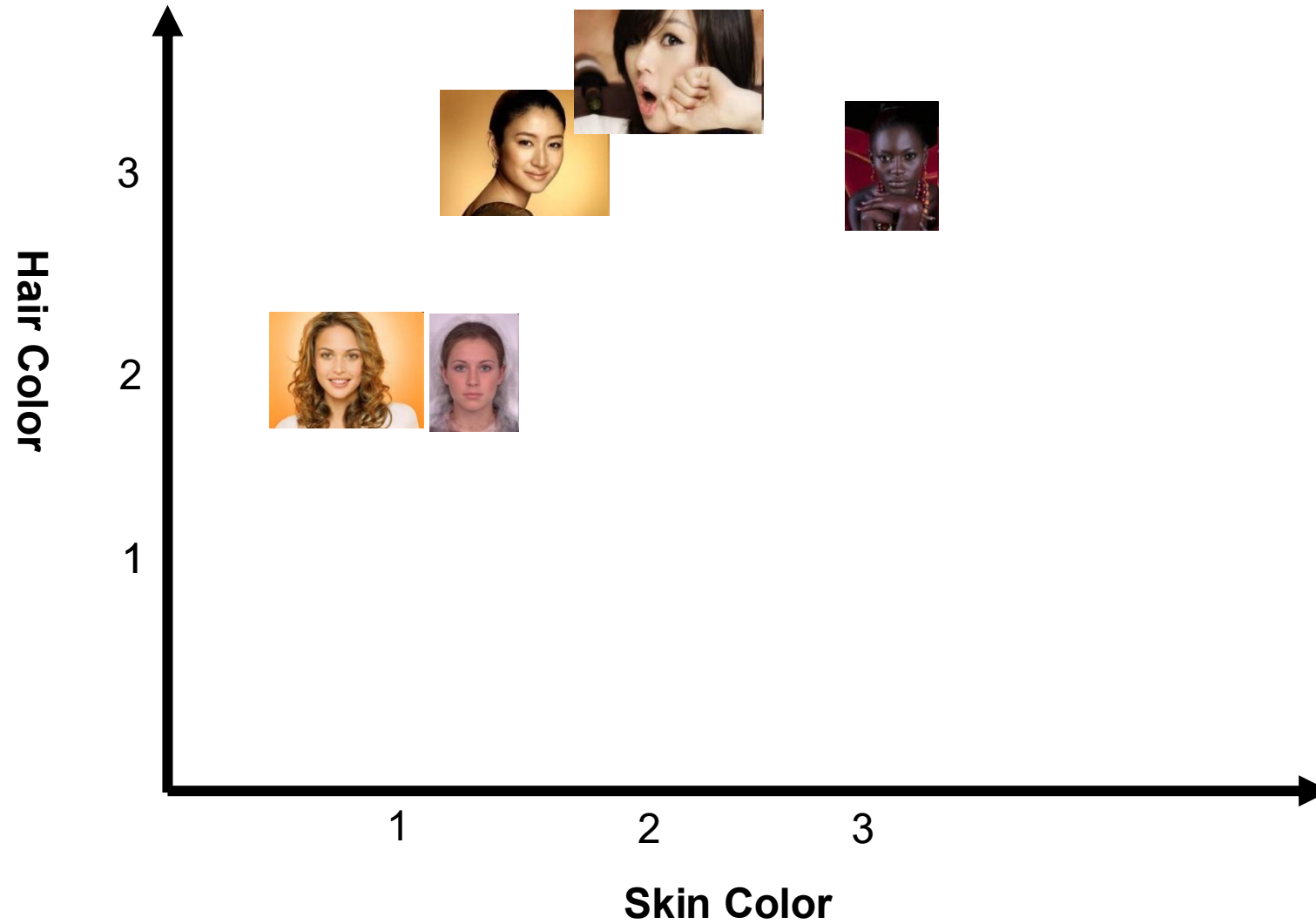


Feature Vectors - Quantization

	Hair Color	Skin Color
	Yellow (2.135)	White (1.05)
	Black (3.324)	Yellow (2.008)
	Black (2.945)	Yellow (2.355)
	Yellow (2.035)	White (1.035)
	Black (2.994)	Black (3.208)

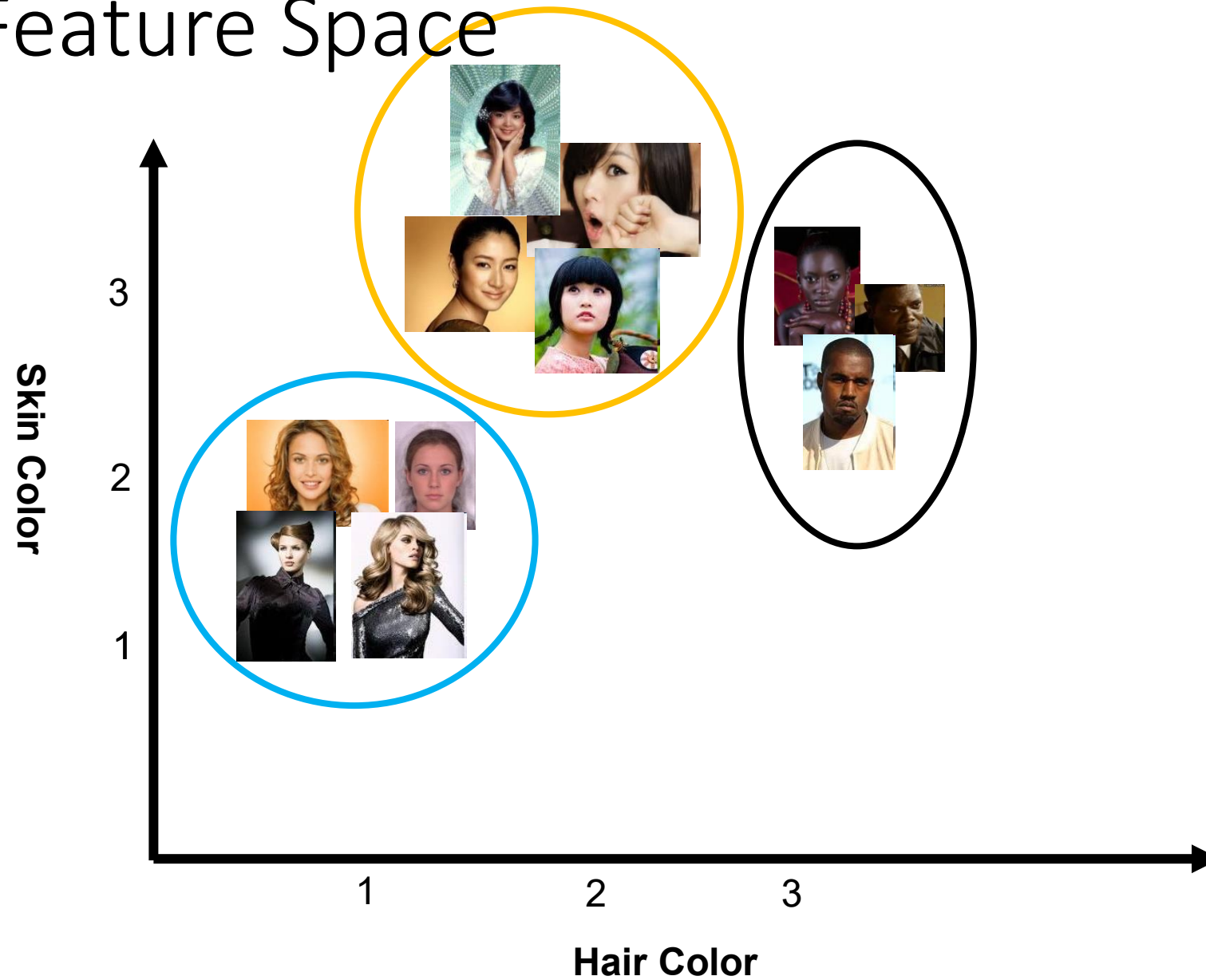


Feature Space





Feature Space





Can we teach computers to draw
circles for grouping?



GOOD GUESS



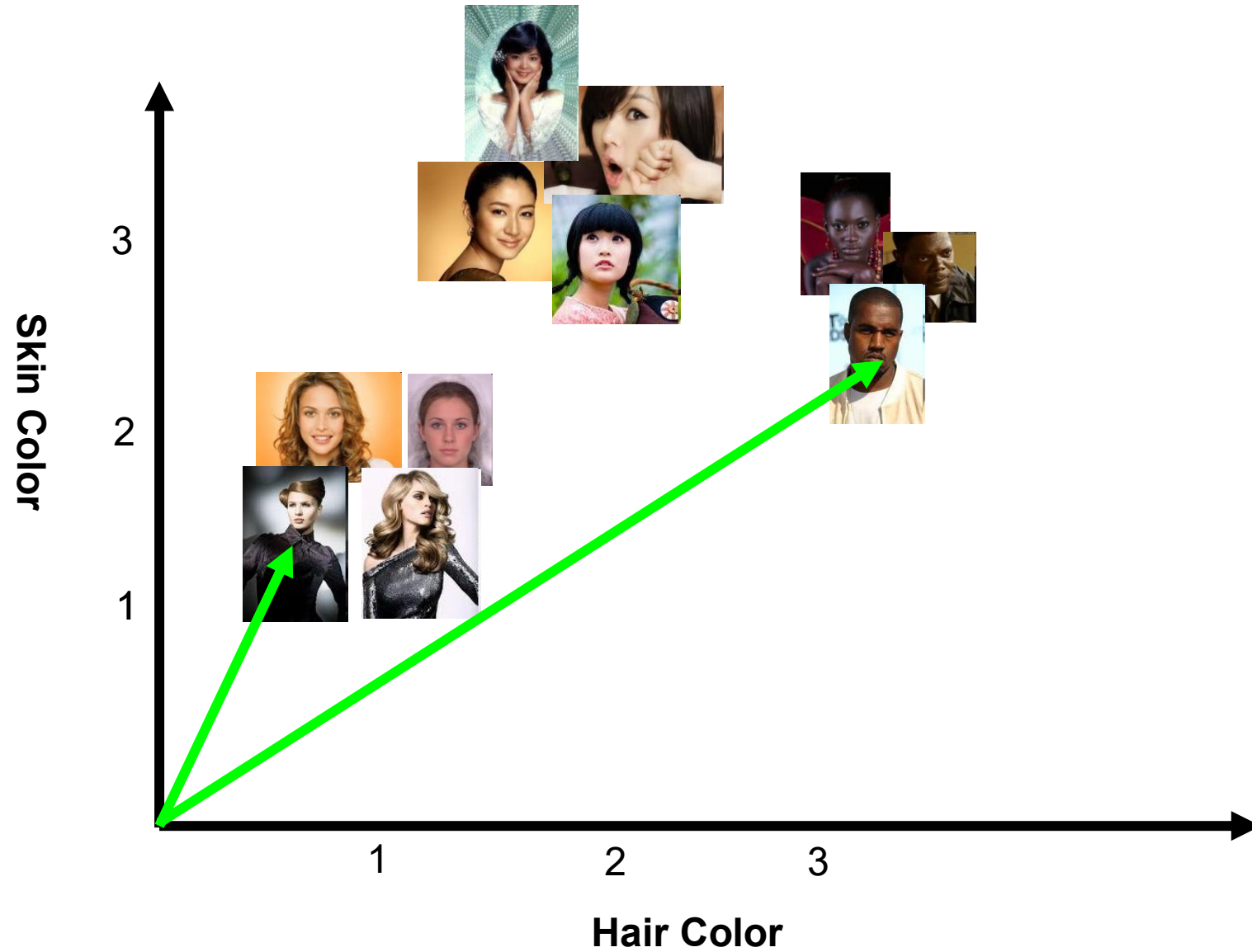


This is what we teach in this course!

But let's start from the most straightforward way by teaching computers to tell the similarity/difference.

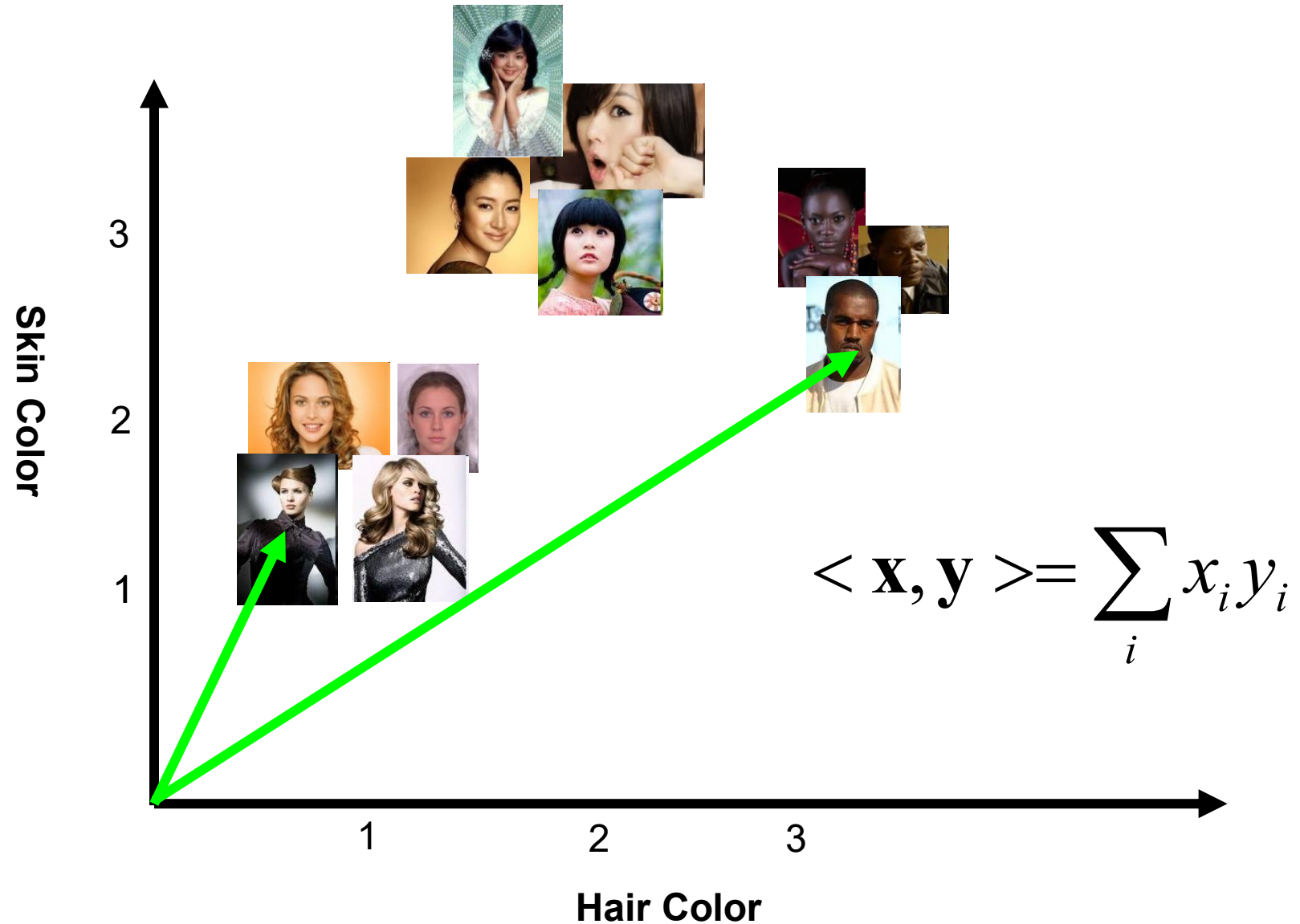


Metrics



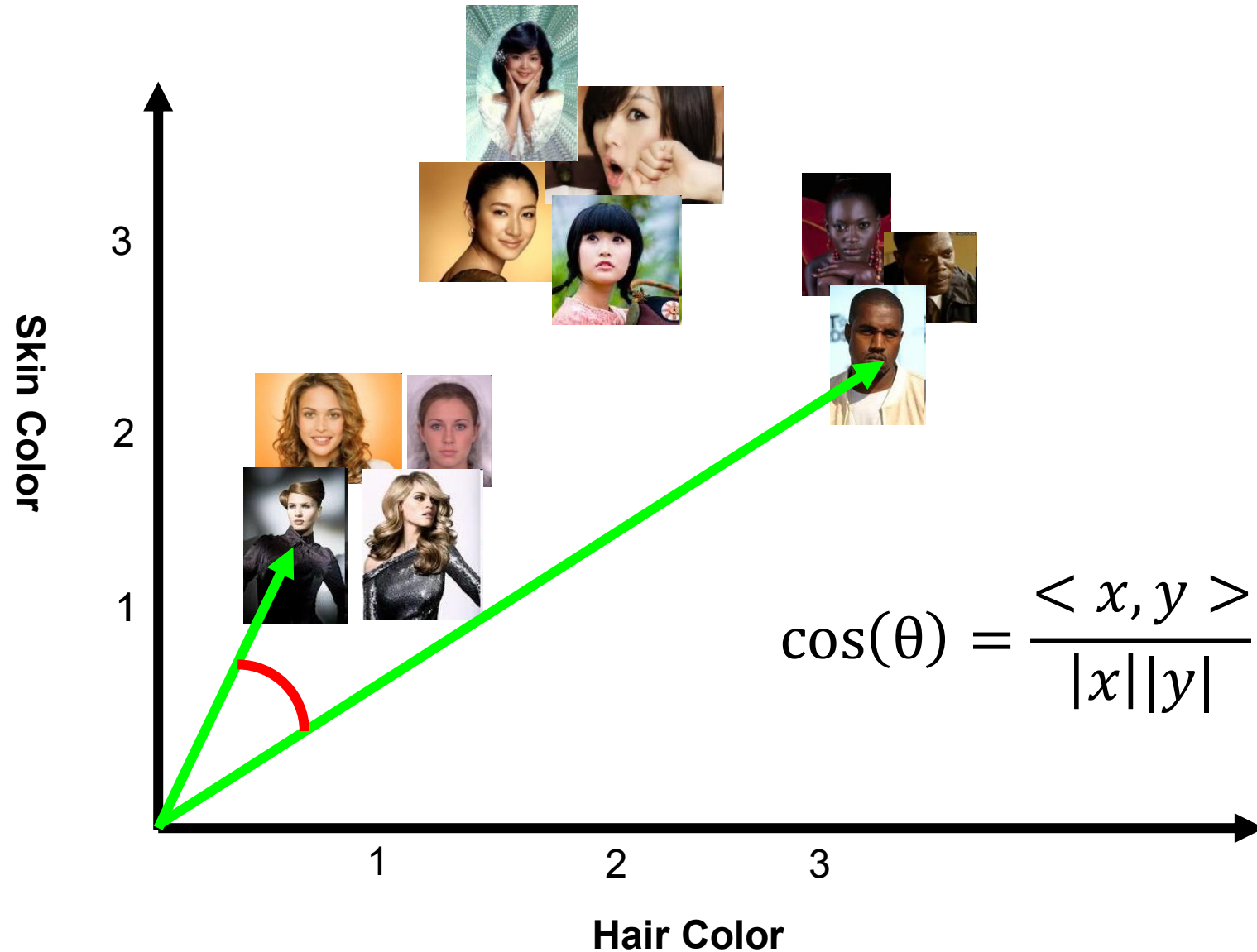


Metrics – Inner Product



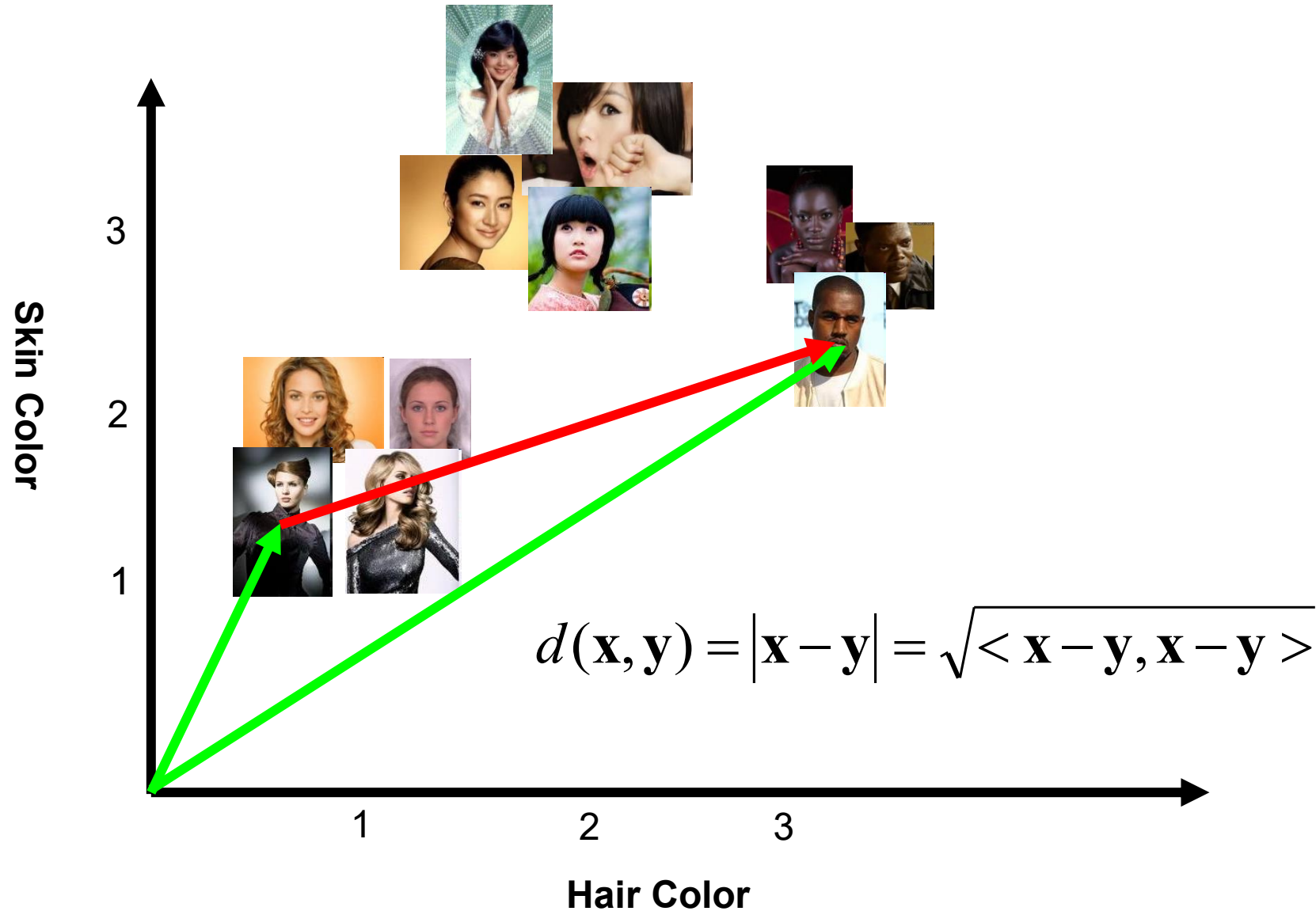


Metrics – Cosine Similarity



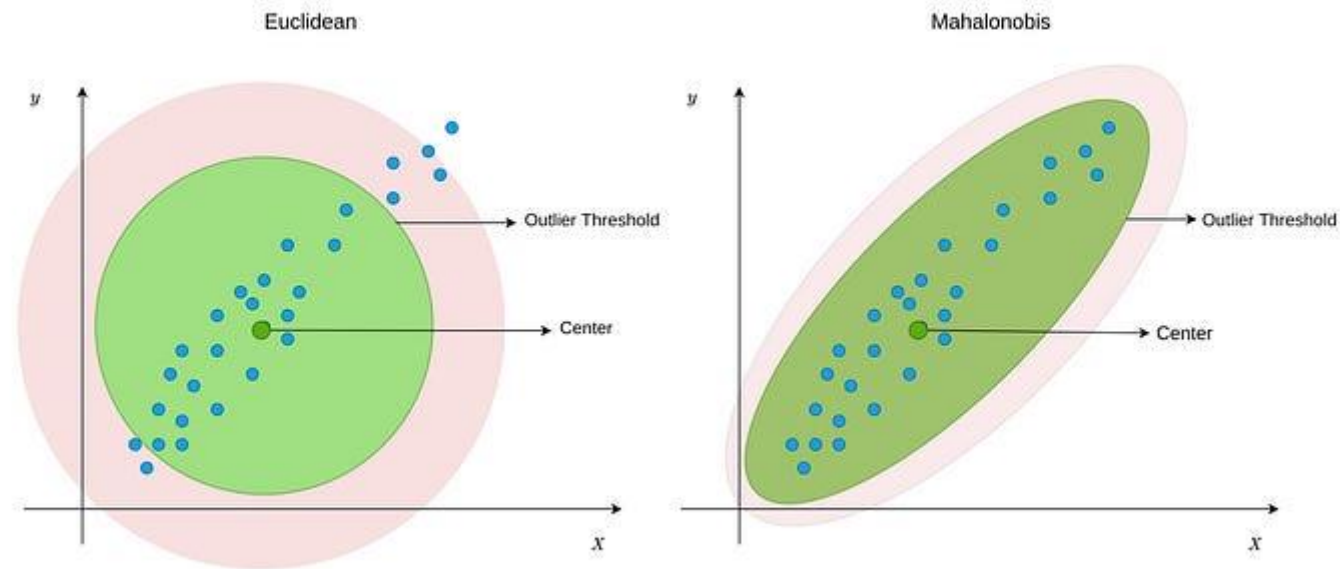


Metrics – Euclidian Distance





Metrics – Mahalanobis Distance



<https://medium.com/data-science/multivariate-outlier-detection-in-python-e946cfc843b3>



I see!

We can tell if two images are from
the same group by measuring the
similarity/distance?



It's too early!



How can we obtain those numbers?

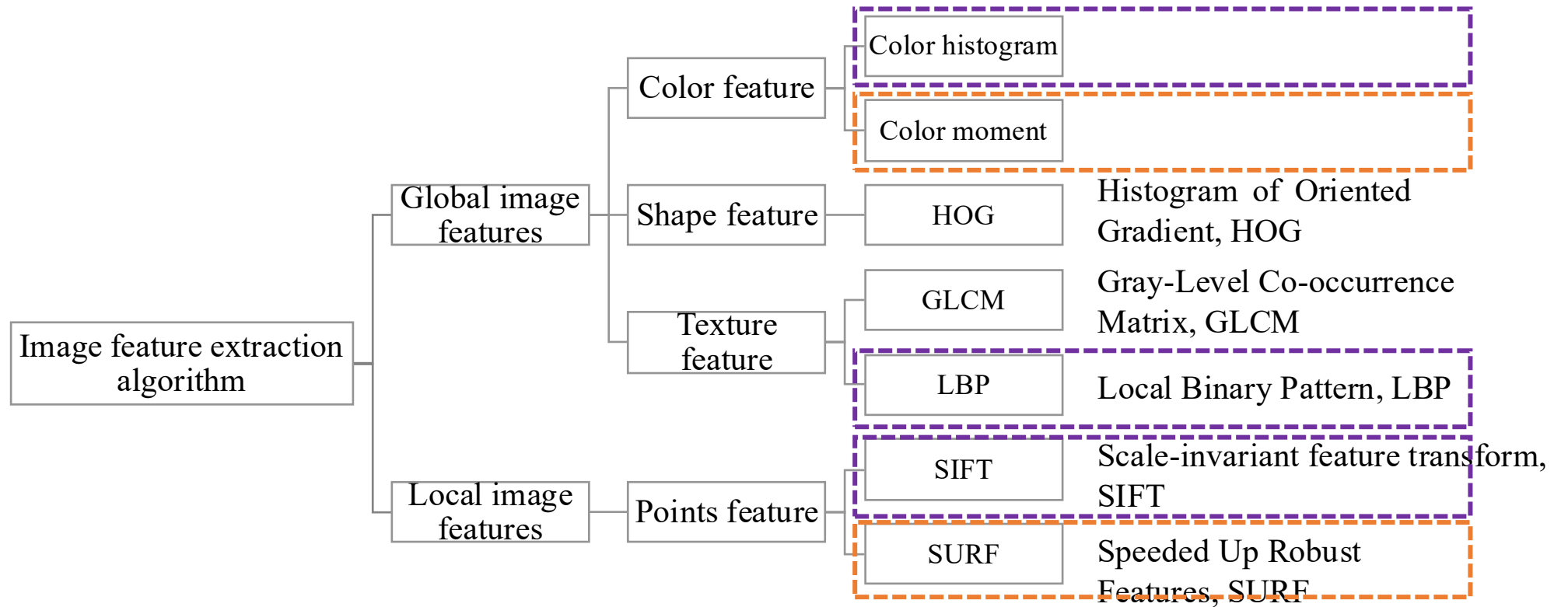
	Hair Color	Skin Color
	Yellow (2.135)	White (1.05)
	Black (3.324)	Yellow (2.008)
	Black (2.945)	Yellow (2.355)
	Yellow (2.035)	White (1.035)
	Black (2.994)	Black (3.208)



Feature Extraction



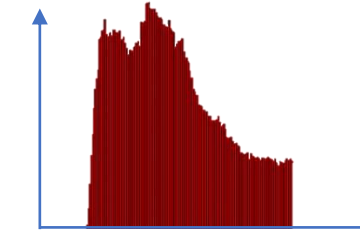
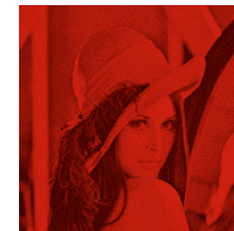
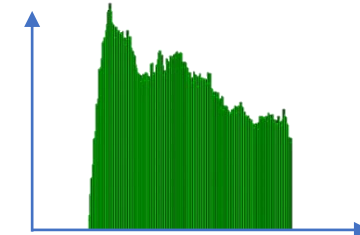
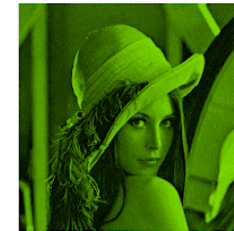
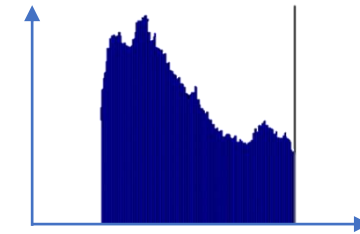
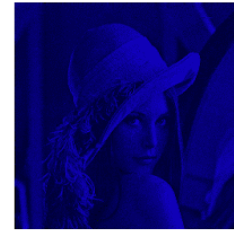
Features





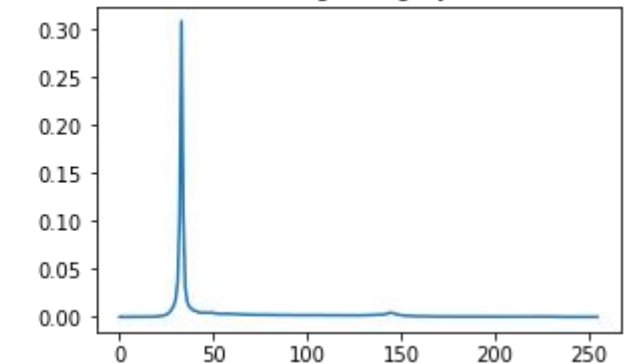
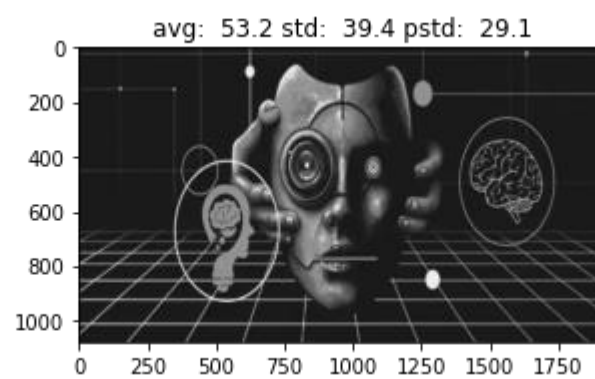
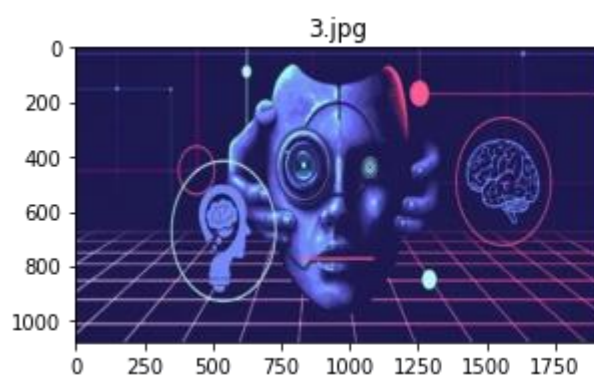
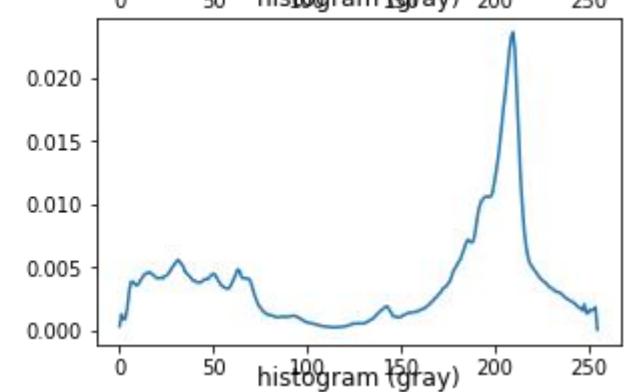
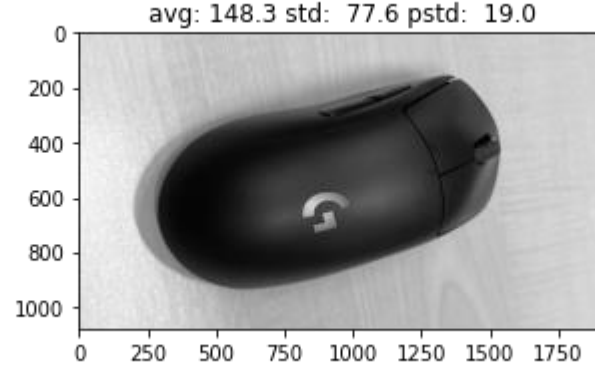
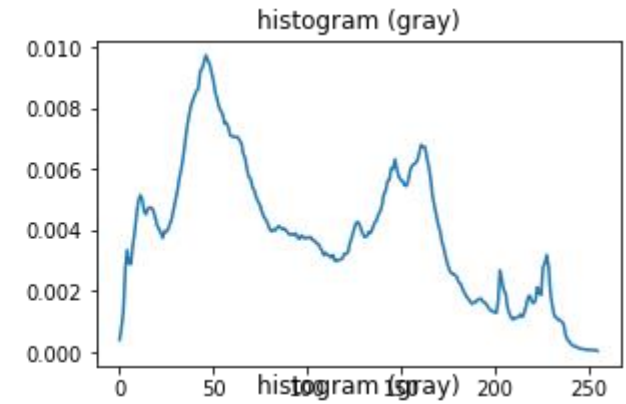
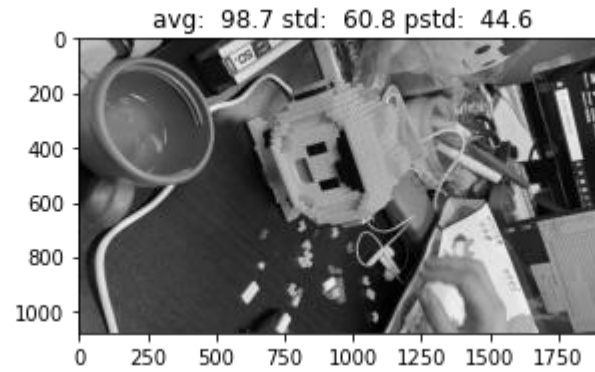
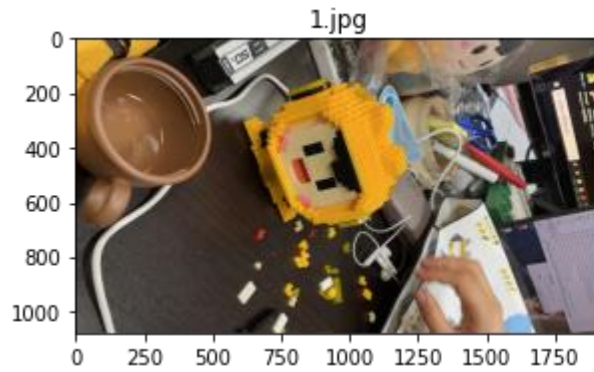
Color Histograms

A color histogram of an image represents the distribution of the composition of colors in the image. It shows different types of colors appeared and the number of pixels in each type of the colors appeared.





Examples from IMHere





Color Histogram Extraction

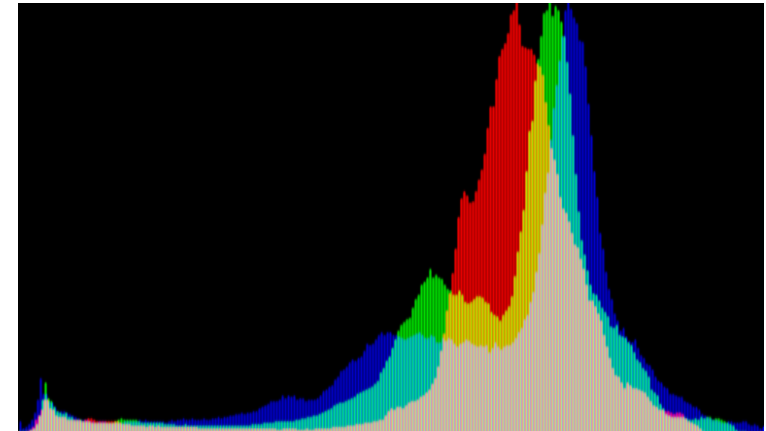
To calculate the color histogram, the color space needs to be divided into several small color intervals, and each cell becomes a bin of the histogram. This process is called color quantization. Then, the color histogram can be obtained by calculating the number of pixels in each cell.



A picture of a cat



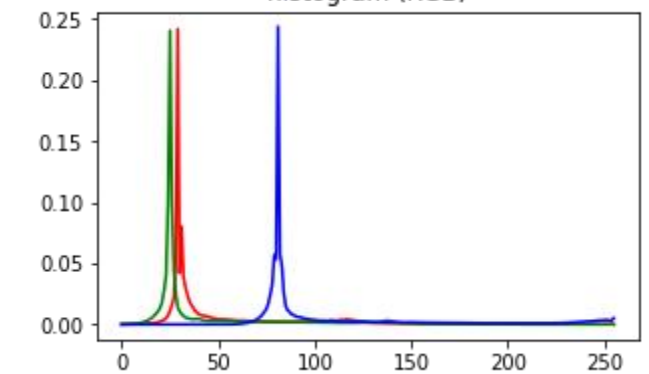
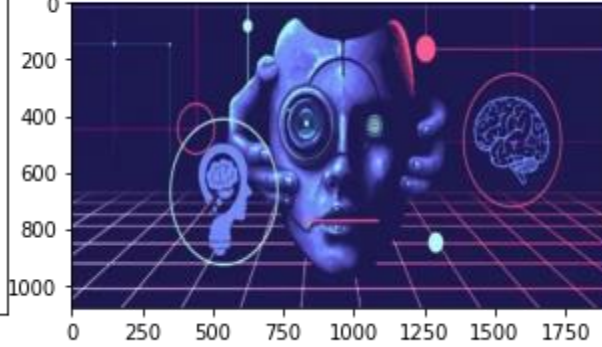
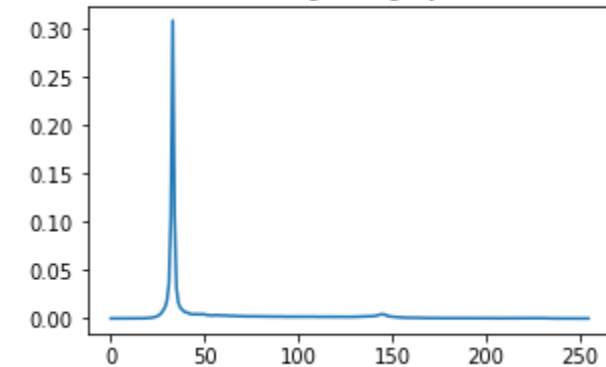
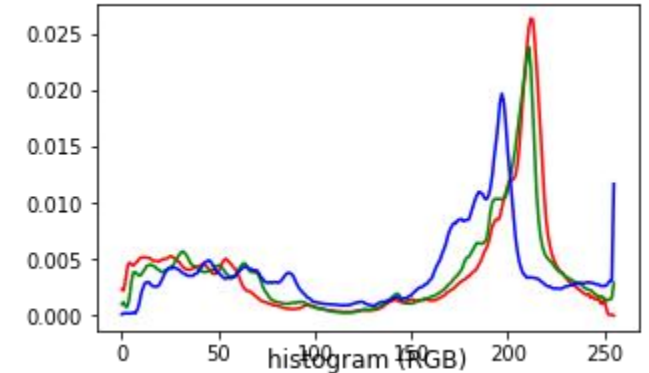
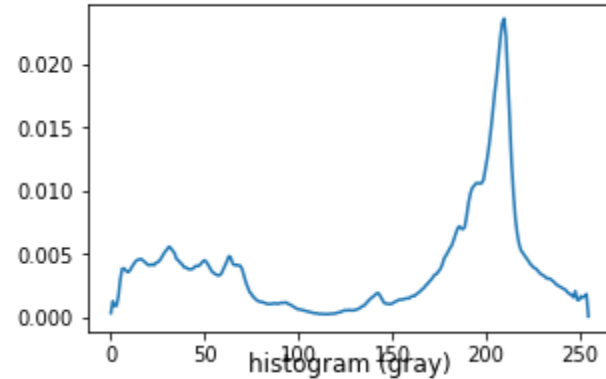
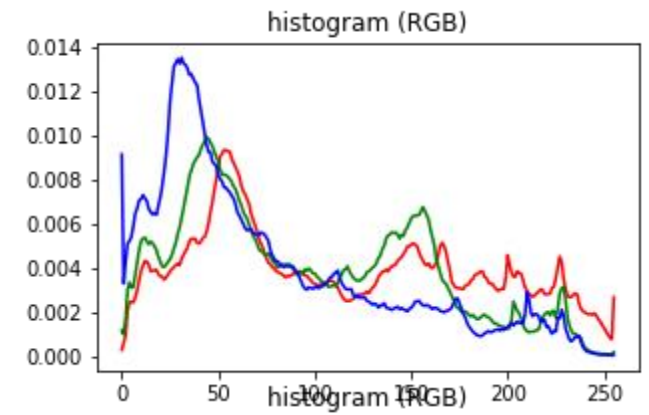
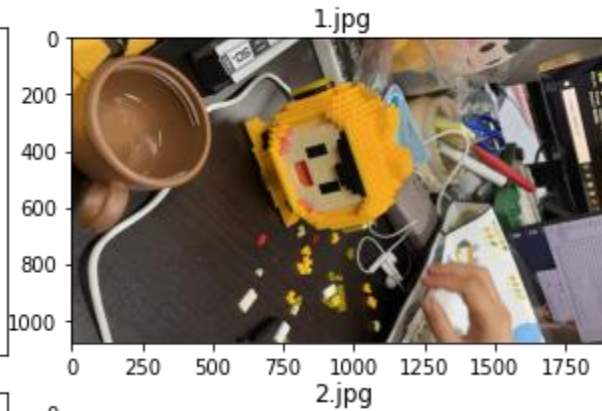
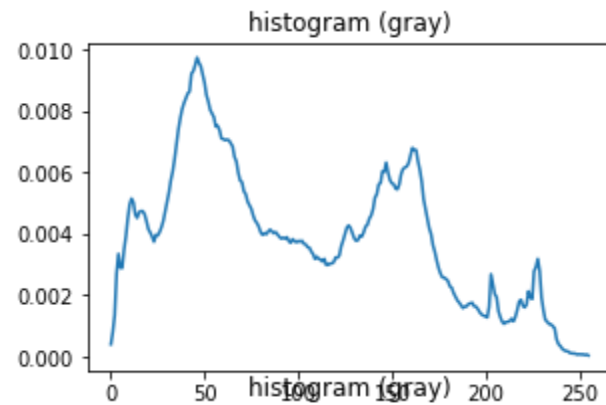
Red	Green	Blue	Pixel Count
Bin 0	Bin 0	Bin 0	7414
Bin 0	Bin 0	Bin 1	230
Bin 0	Bin 0	Bin 2	0
Bin 0	Bin 0	Bin 3	0
Bin 0	Bin 1	Bin 0	8
Bin 0	Bin 1	Bin 1	372
Bin 0	Bin 1	Bin 2	88
Bin 0	Bin 1	Bin 3	0
Bin 0	Bin 2	Bin 0	0
Bin 0	Bin 2	Bin 1	0
Bin 0	Bin 2	Bin 2	10
Bin 0	Bin 2	Bin 3	1
Bin 0	Bin 3	Bin 0	0
Bin 0	Bin 3	Bin 1	0



Color histogram of the cat picture with x-axis being RGB and y-axis being the frequency.

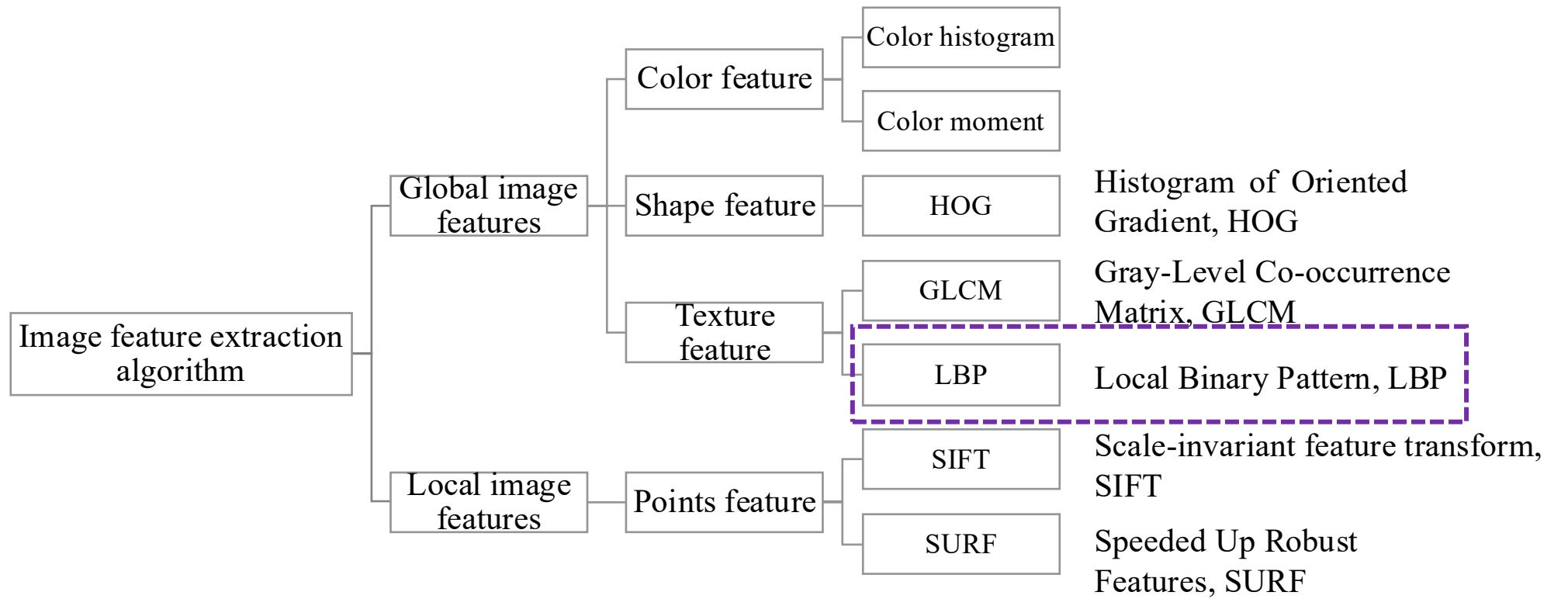


Examples from IMHere





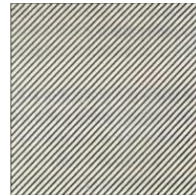
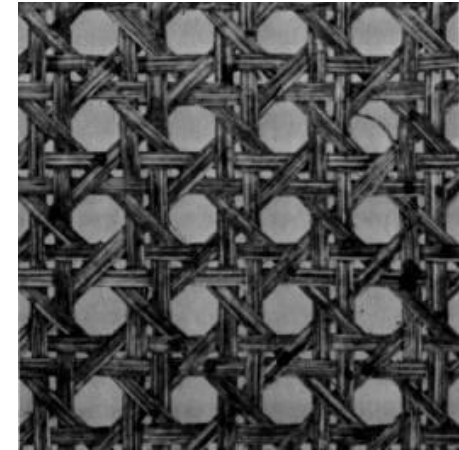
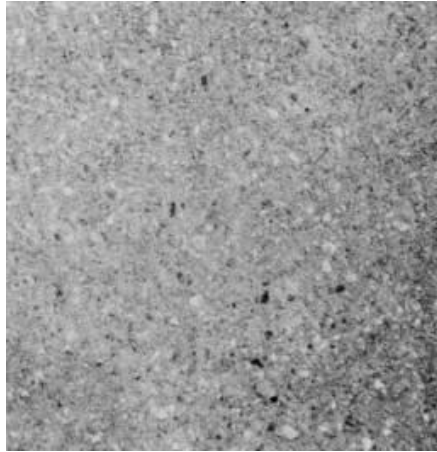
Features





Texture Features

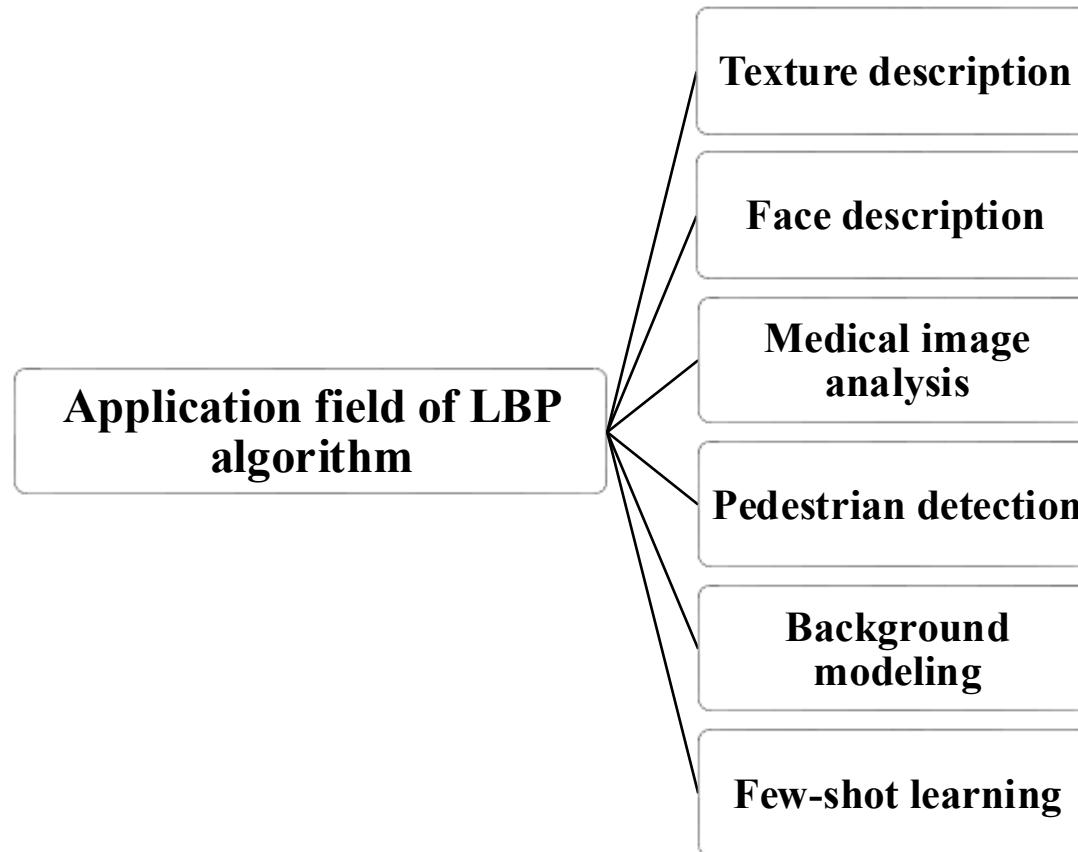
Texture feature refers to the common internal characteristics of the object surface, which contains the important information of the structure and arrangement of the object surface and its relationship with the surrounding objects.





Local Binary Patterns (LBP)

LBP algorithm was originally proposed by Timo Ojala and Matti Pietikäinen of machine vision and signal analysis center of Oulu University in Finland in 1996 for texture feature description.



Matti Pietikäinen
IEEE、IAPR Fellow
CMVSA of Oulu University



Local Binary Patterns (LBP)



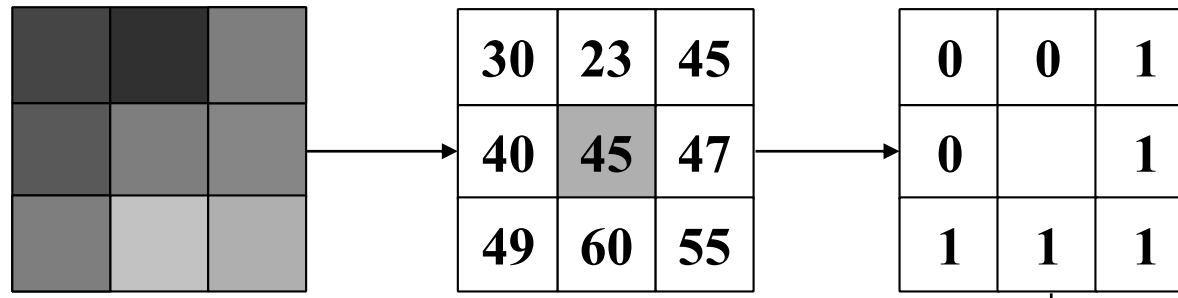
Gray-scale image



LBP feature map



Local Binary Patterns (LBP)



0	0	1	1	1	1	1	0
---	---	---	---	---	---	---	---

1	2	4	8	16	32	64	128
---	---	---	---	----	----	----	-----

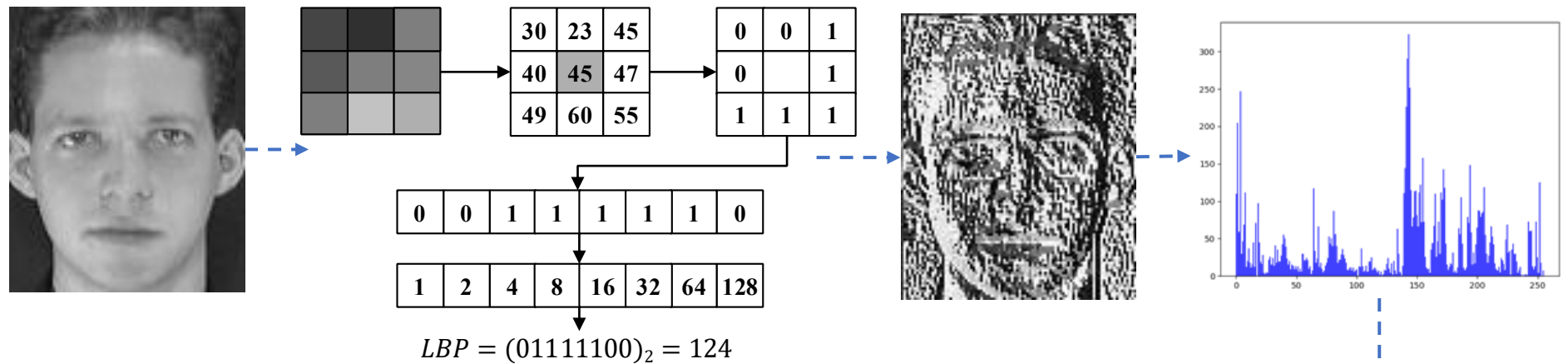
$$LBP = (01111100)_2 = 124$$

$$LBP_{P,R} = \sum_{p=1}^P S(g_p - g_c) \times 2^{p-1}$$

$$S(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



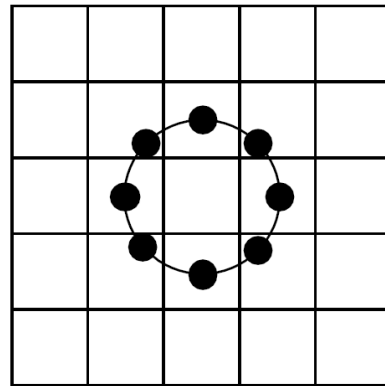
Local Binary Patterns (LBP)



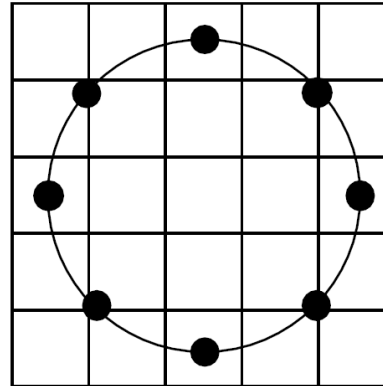
Feature vector $X = [x_1, x_2, x_3, \dots, x_K]$



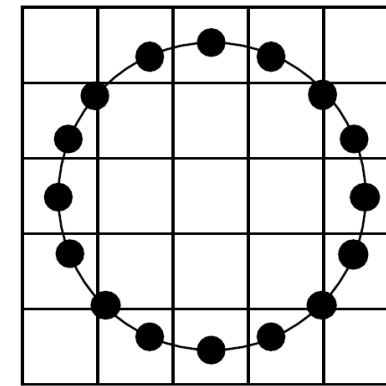
Circular LBP



R=1 P=8



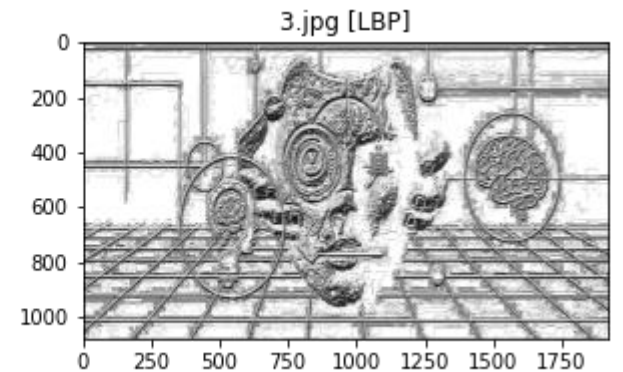
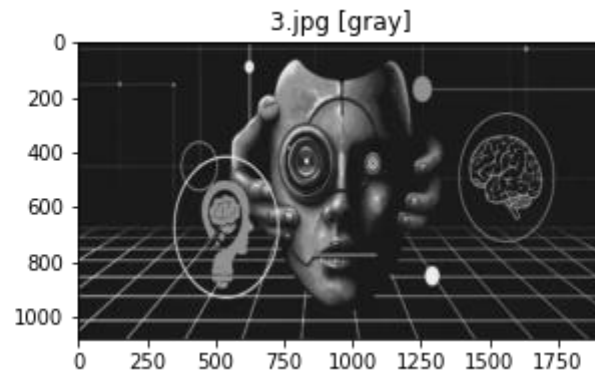
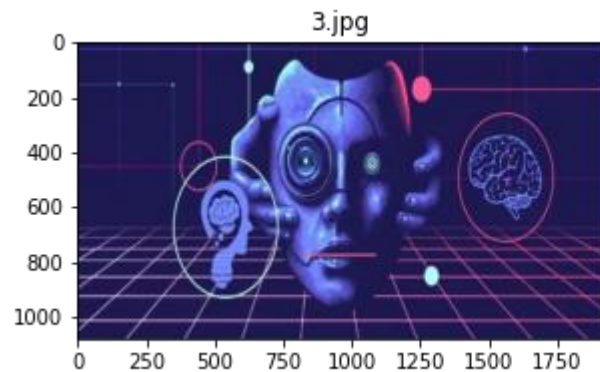
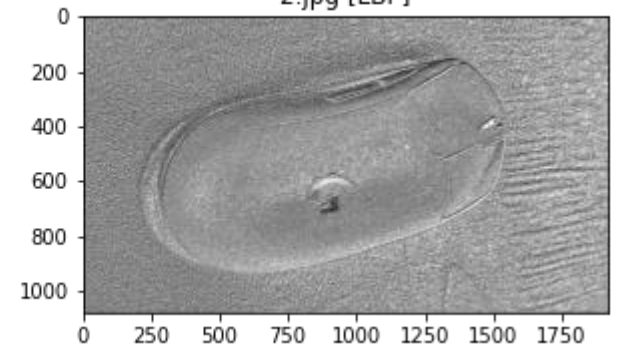
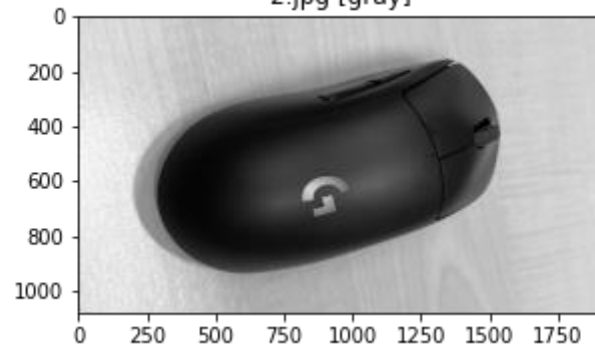
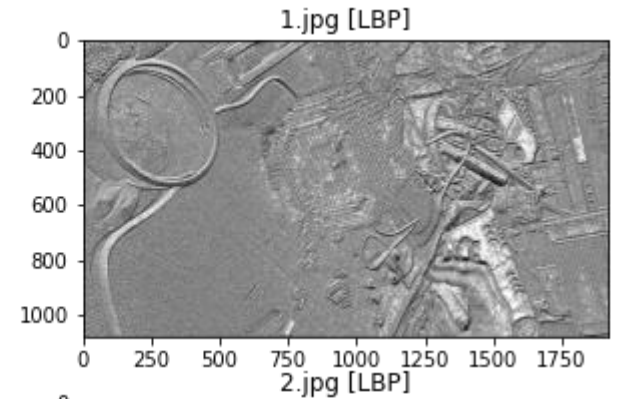
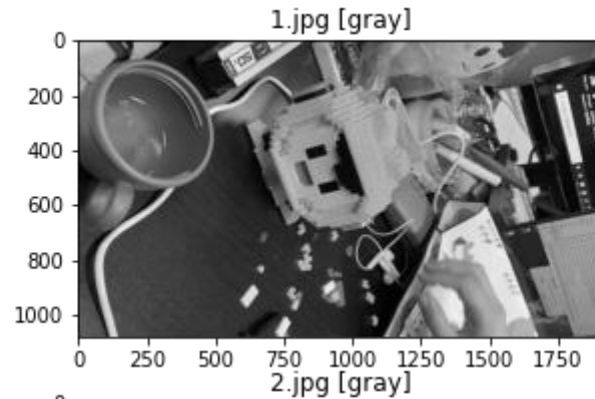
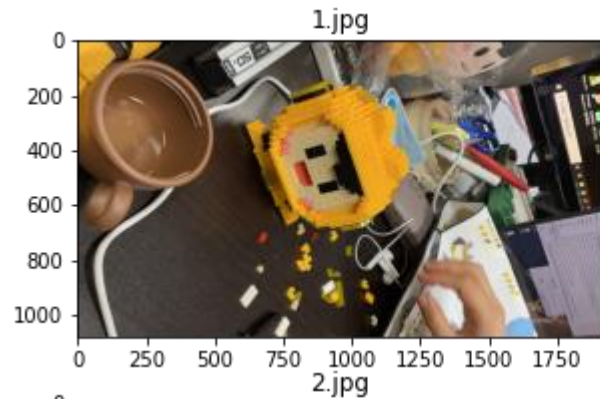
R=2 P=8



R=2 P=16



Examples from IMHere



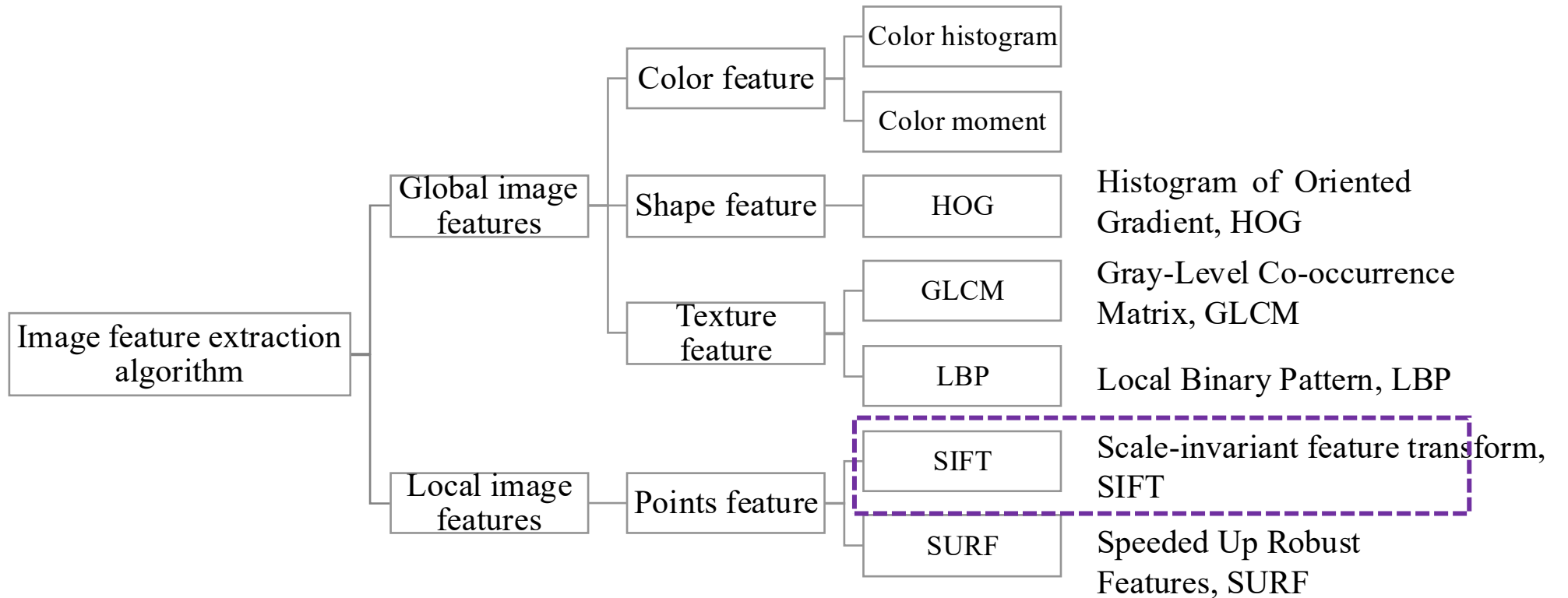


Can computers “see” the difference using color histograms or LBP?





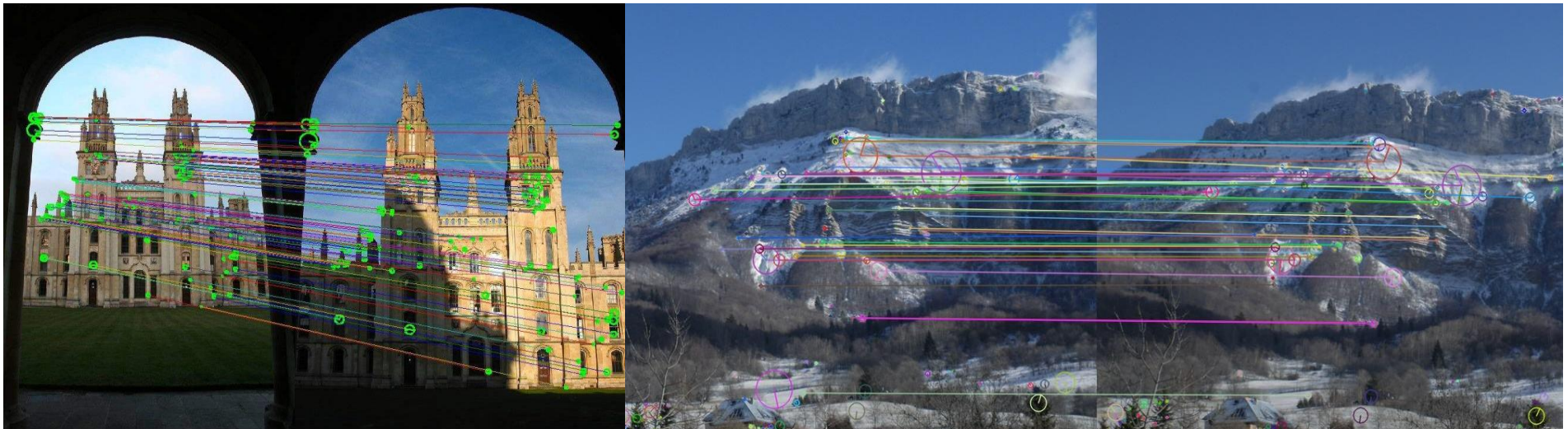
Features





Scale-invariant feature transform (SIFT)

SIFT is a milestone work in the research field of local image feature descriptor. SIFT was first proposed in 1999 and was improved in 2004. SIFT is invariant to image changes such as scale, rotation, certain viewing angle and illumination changes.



<http://5b0988e595225.cdn.sohucs.com/images/20200318/bacc3866c2eb47b58cde6afd3b6512b6.jpeg>

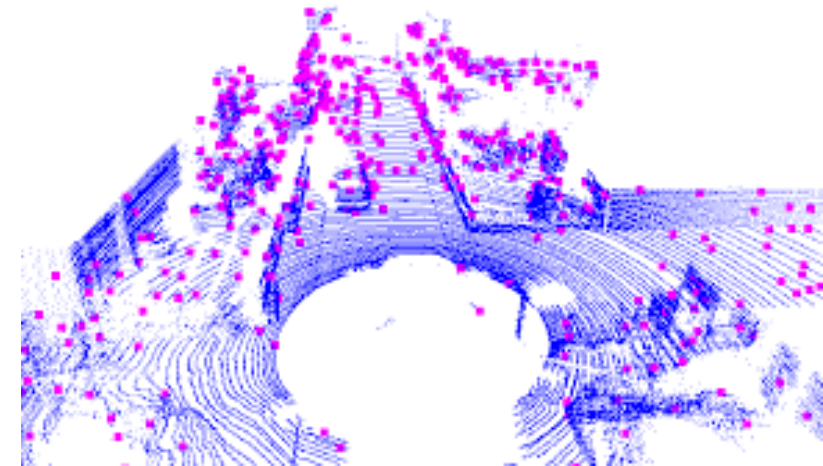
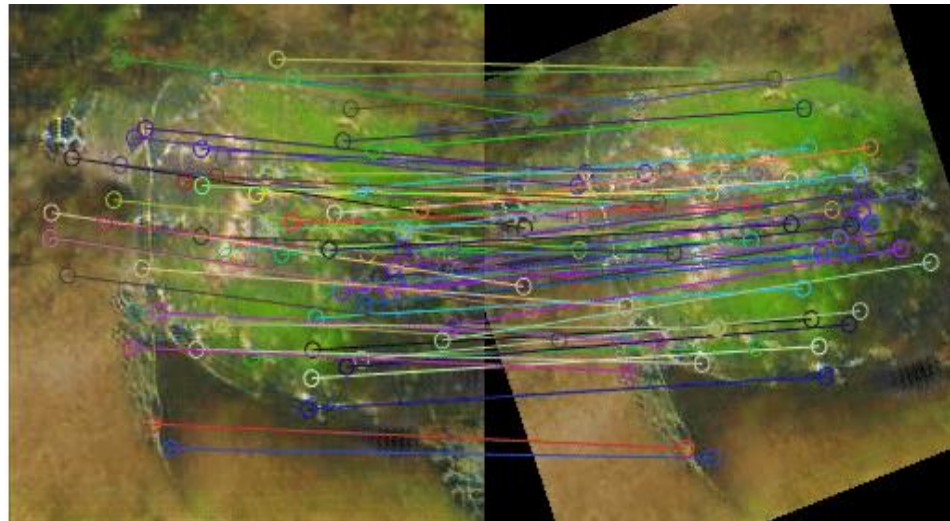
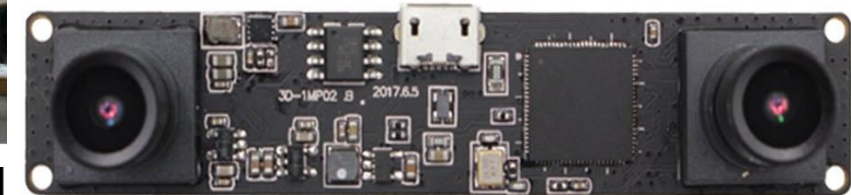


Scale-invariant feature transform (SIFT)



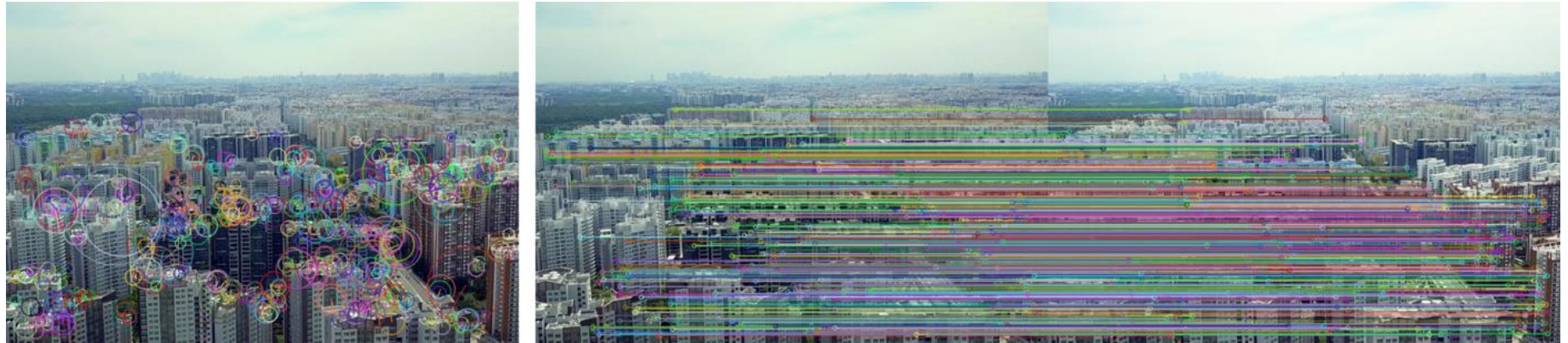


Scale-invariant feature transform (SIFT)





Scale-invariant feature transform (SIFT)





Scale-invariant feature transform (SIFT)



Mining In-class Social Networks for Large-scale Pedagogical Analysis,
ACM International Conference on Multimedia, 2012



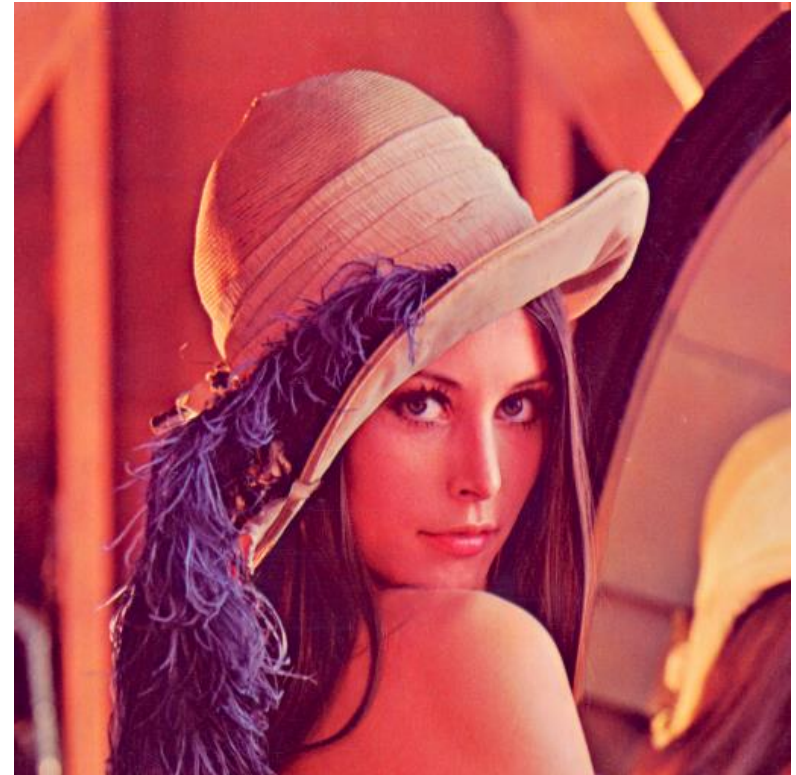
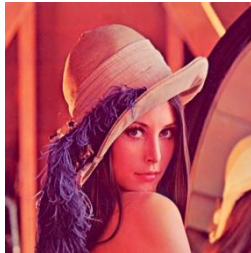
SIFT Detection & Description Steps

- **Candidate Localization:** Find Scale-invariant candidates
- **Refinement:** Keypoint filtering
- **Orientation Assignment:** Estimate orientations for keypoints
- **Description Generation:** Build description



Step 1: Candidate Localization

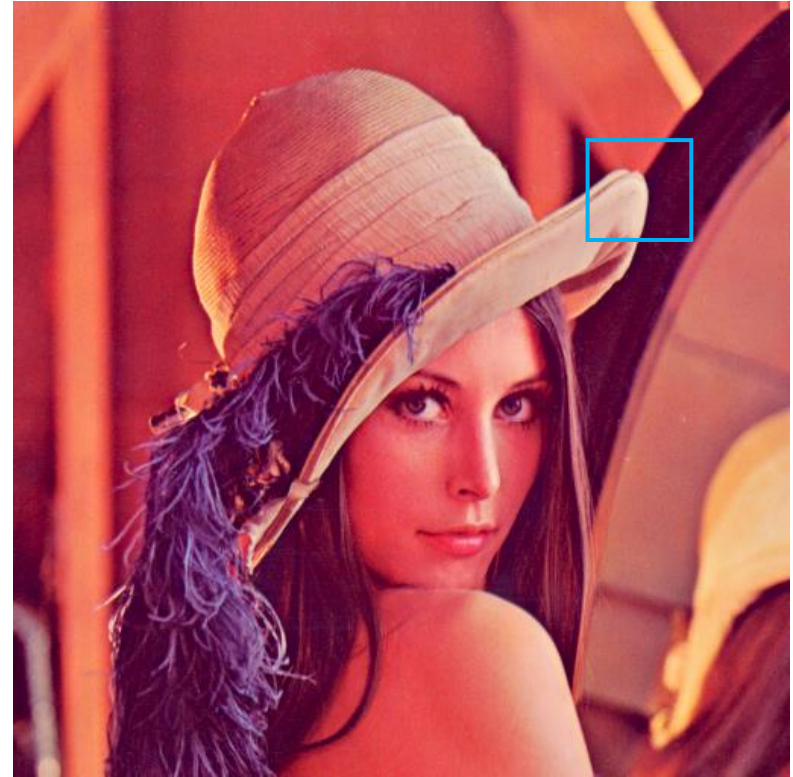
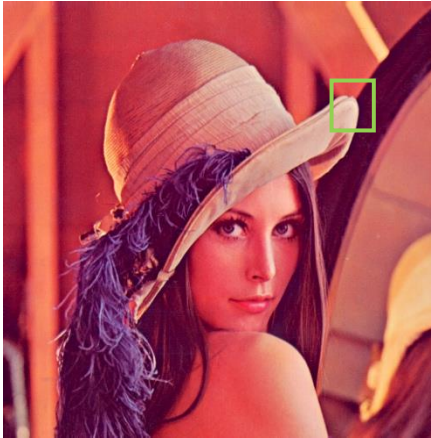
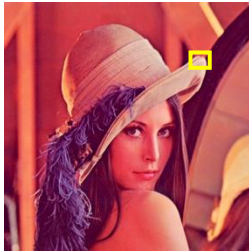
- What is a scale?





Step 1: Candidate Localization

- What is a scale?





Step 1: Candidate Localization

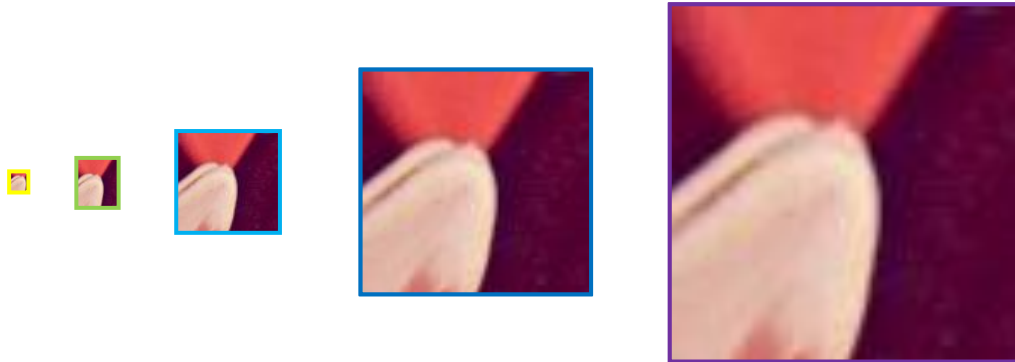
- What is a scale?





Step 1: Candidate Localization

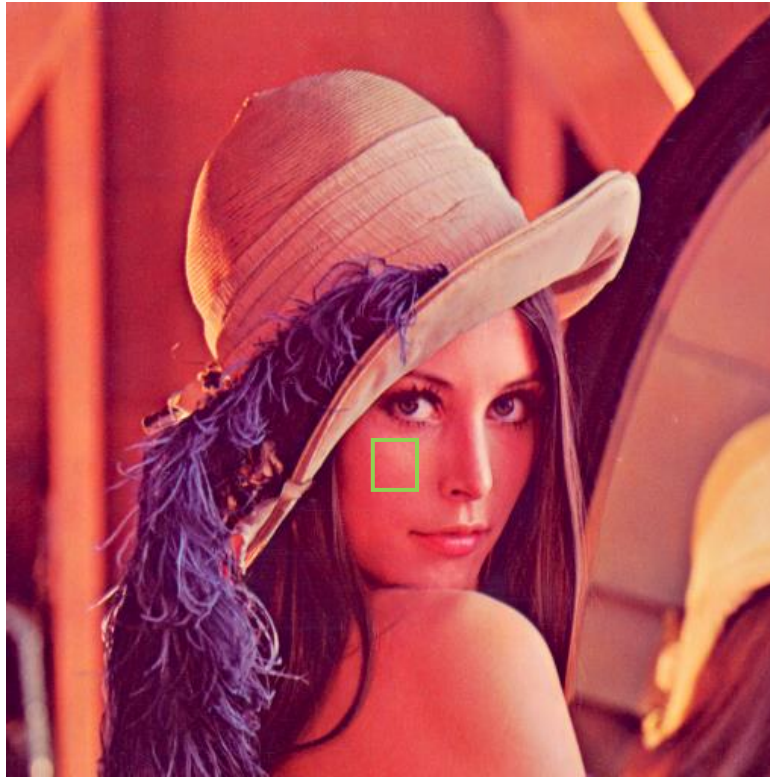
- What is a scale?





Step 1: Candidate Localization

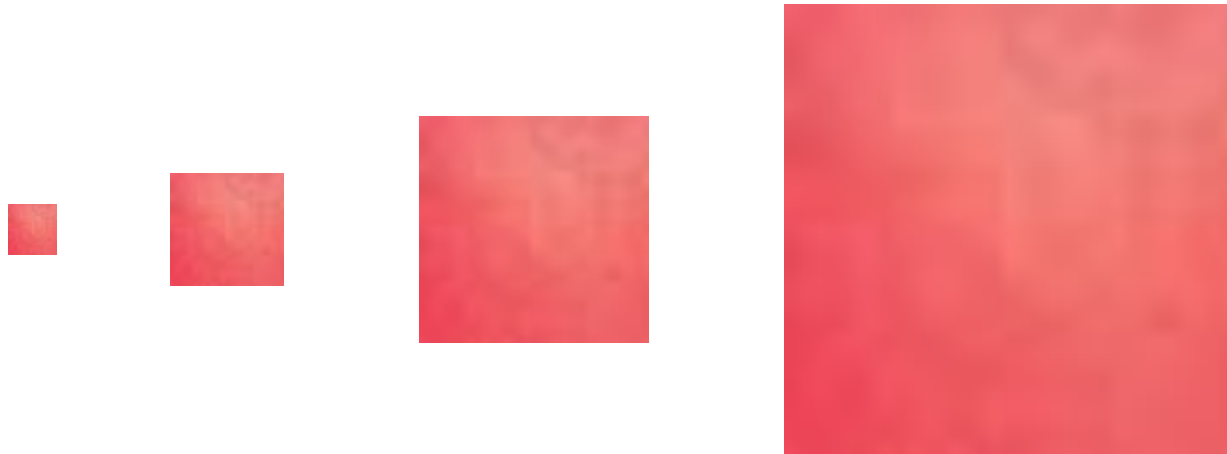
- What is a scale?





Step 1: Candidate Localization

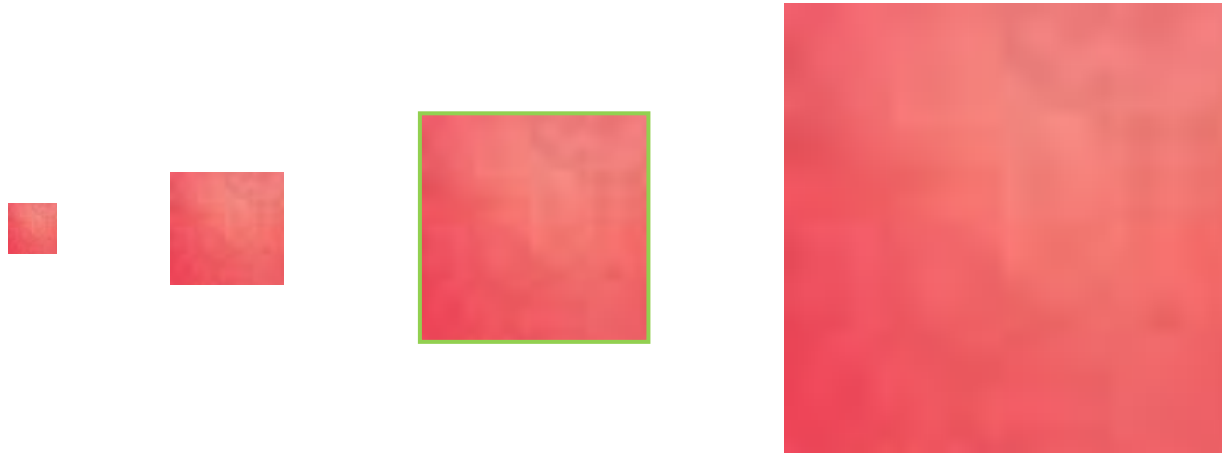
- What is scale-invariant?





Step 1: Candidate Localization

- What is scale-invariant?

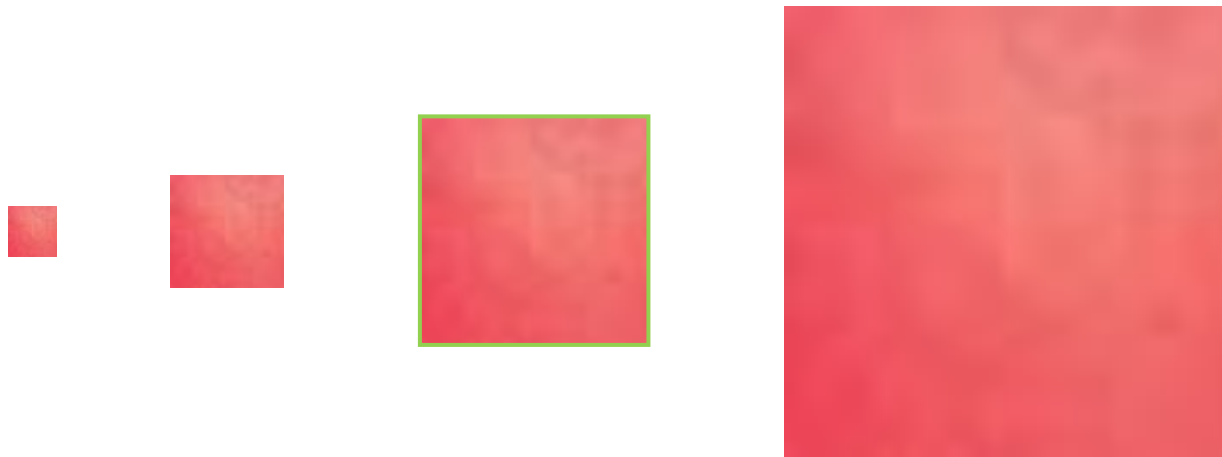


To find the scale at which a pattern is represented the **best** at its center (the key point)



Step 1: Candidate Localization

- What is scale-invariant?



Scale-invariance: no matter what the original scale of the image is, the same **best scale** will be found

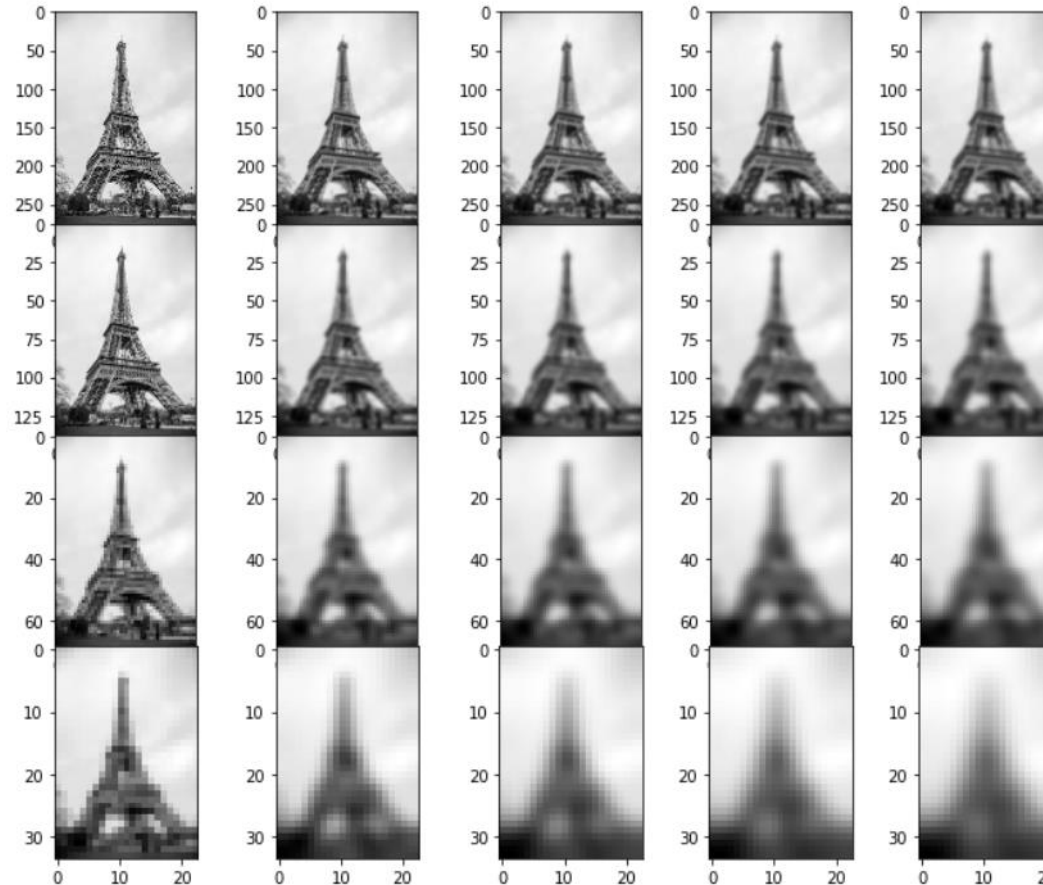


How can we do it?



Step 1: Candidate Localization

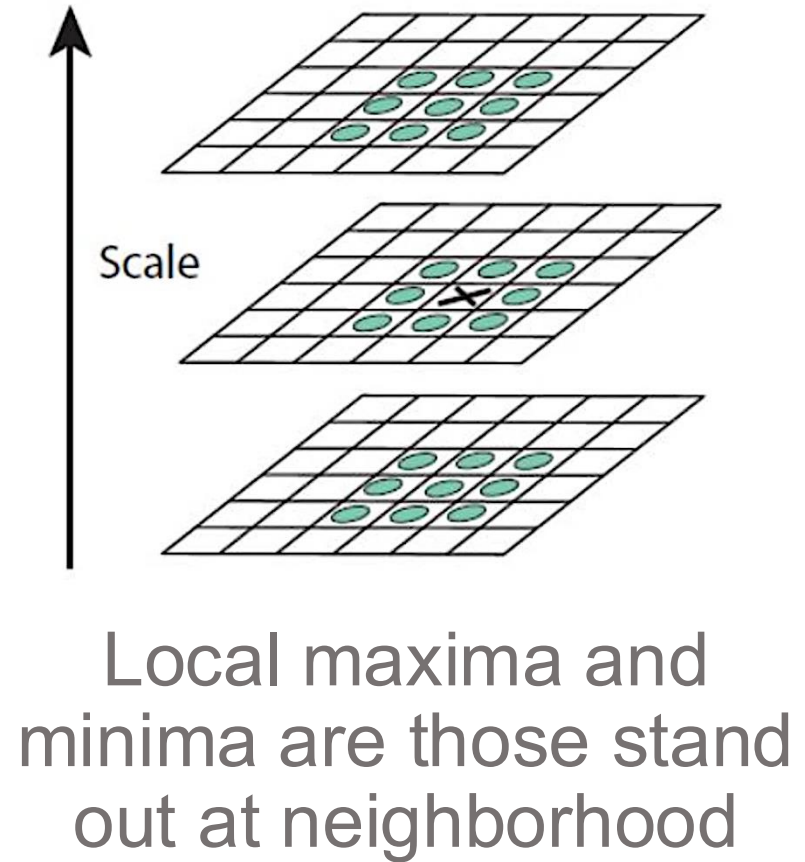
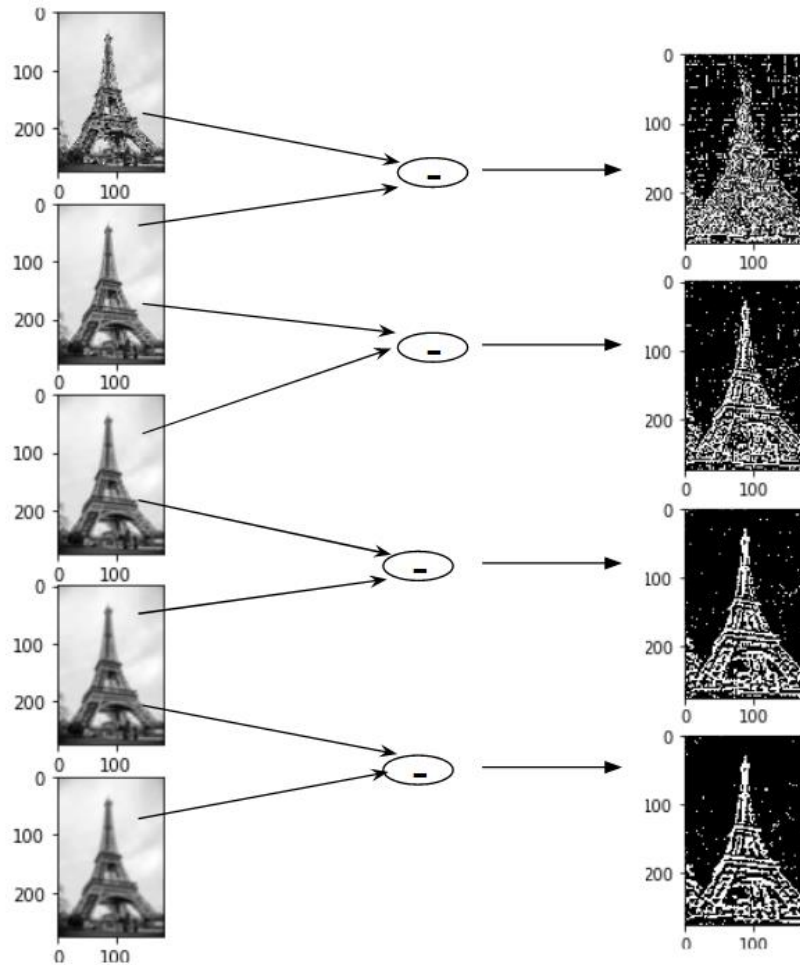
- Rescale and blur





Step 1: Candidate Localization

- Difference of Gaussian



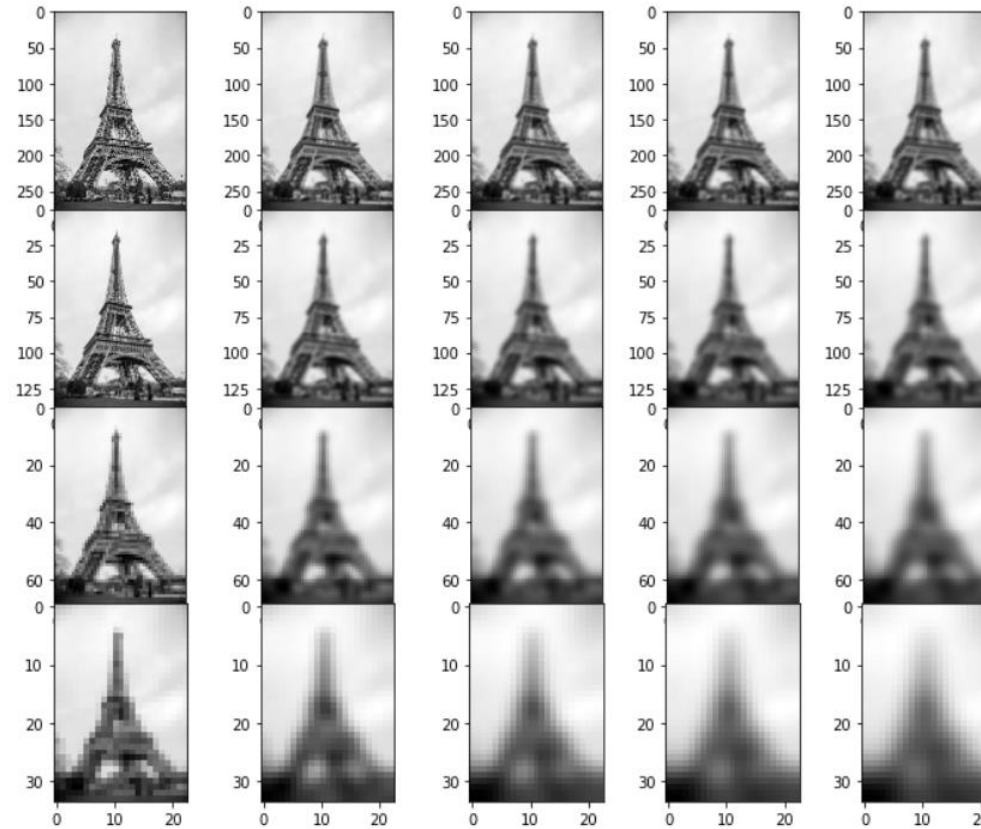


Sounds good but less efficient. We
need a little math here.



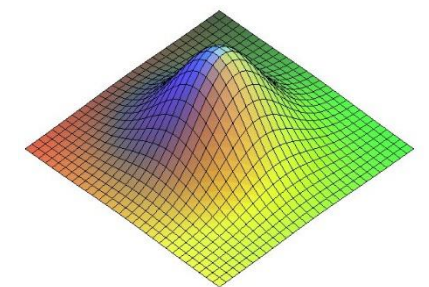
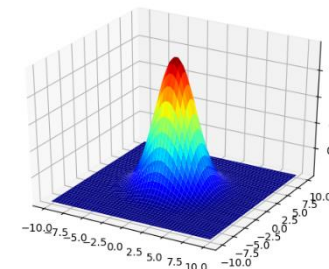
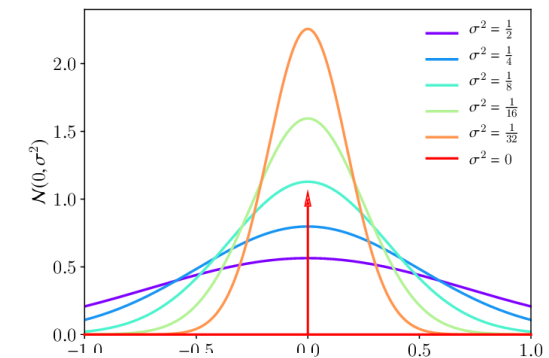
Step 1: Candidate Localization

- Rescale and blur



$$G(x, y, \sigma) * I(x, y)$$

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

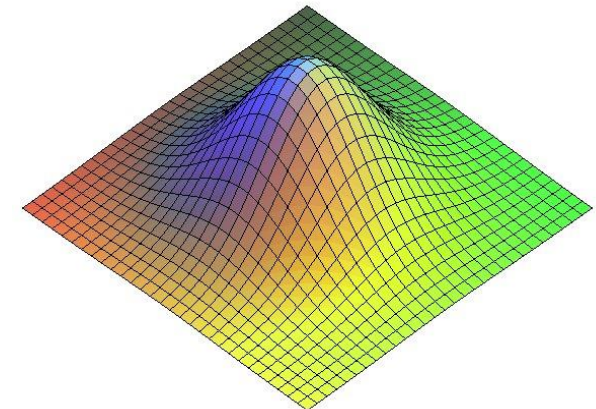
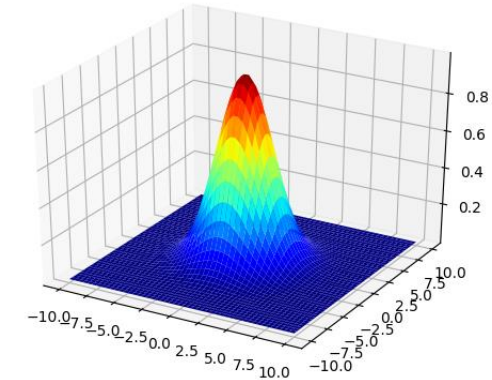




Step 1: Candidate Localization

0.0049	0.0092	0.0134	0.0152	0.0134	0.0092	0.0049
0.0092	0.0172	0.0250	0.0283	0.0250	0.0172	0.0092
0.0134	0.0250	0.0364	0.0412	0.0364	0.0250	0.0134
0.0152	0.0283	0.0412	0.0467	0.0412	0.0283	0.0152
0.0134	0.0250	0.0364	0.0412	0.0364	0.0250	0.0134
0.0092	0.0172	0.0250	0.0283	0.0250	0.0172	0.0092
0.0049	0.0092	0.0134	0.0152	0.0134	0.0092	0.0049

This is a 7 x 7 (truncated) Gaussian mask with a mean of zero and a standard deviation $\sigma = 2$





Step 1: Candidate Localization



Scale = 0



Scale = 1



Scale = 4



Scale = 16



Scale = 64

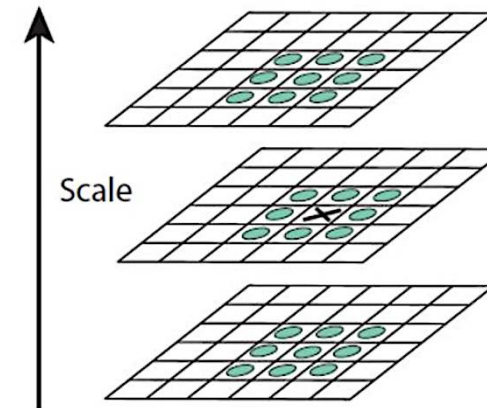
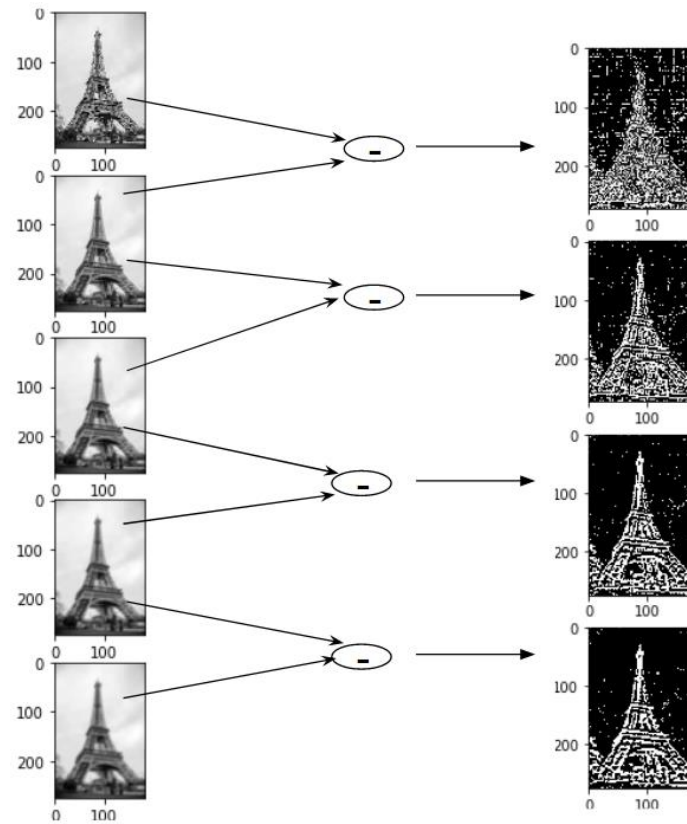


Scale = 256

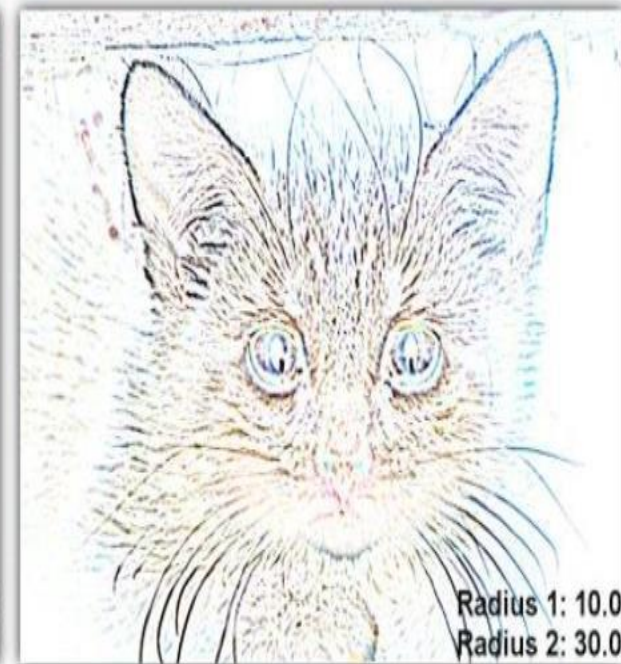
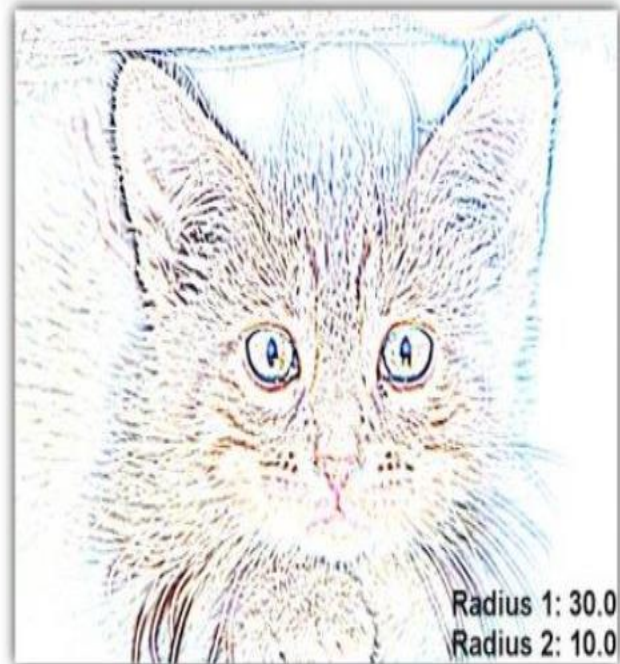
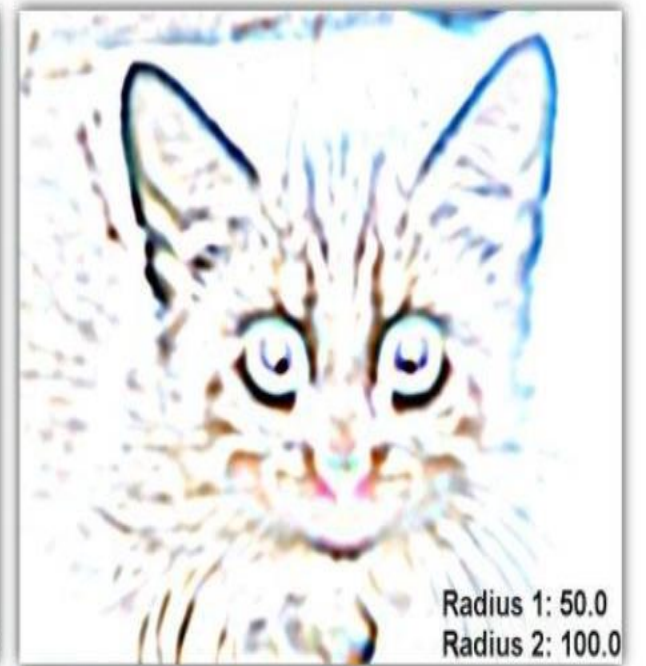
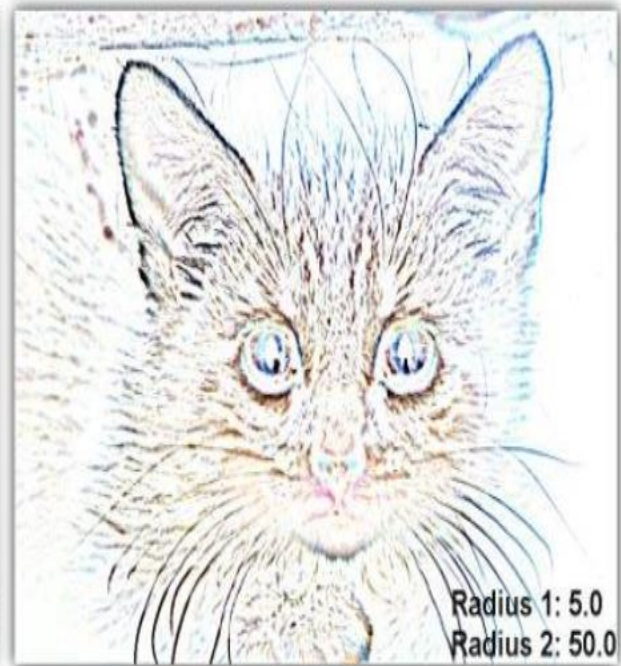
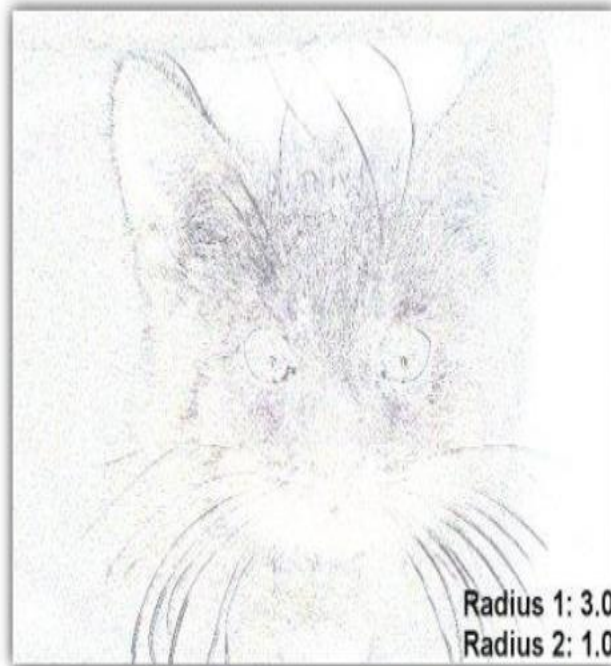


Step 1: Candidate Localization

- Difference of Gaussian (DoG)

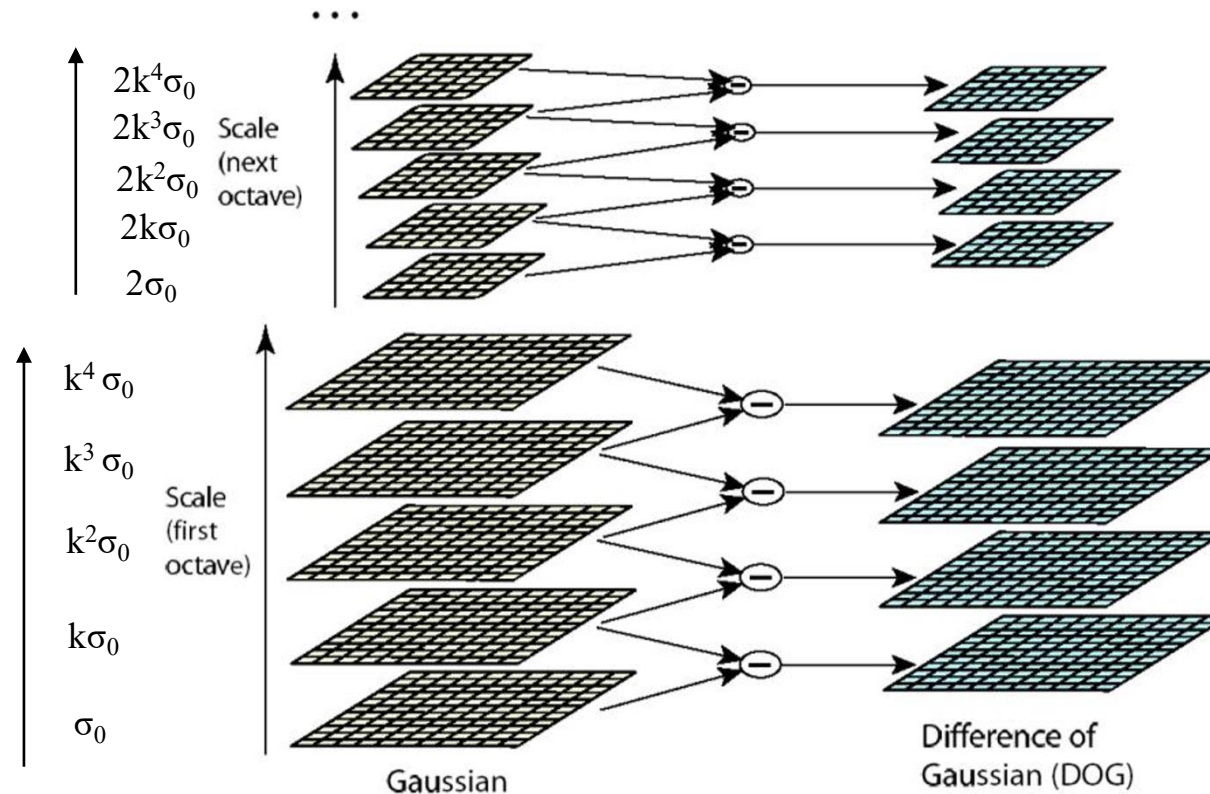


$$\begin{aligned} L(x, y, \sigma) &= G(x, y, \sigma) * I(x, y) \\ D(x, y, \sigma) &= L(x, y, \sigma) - L(x, y, k\sigma) \\ &= (G(x, y, \sigma) - G(x, y, k\sigma)) * I(x, y) \end{aligned}$$





Step 1: Candidate Localization



$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

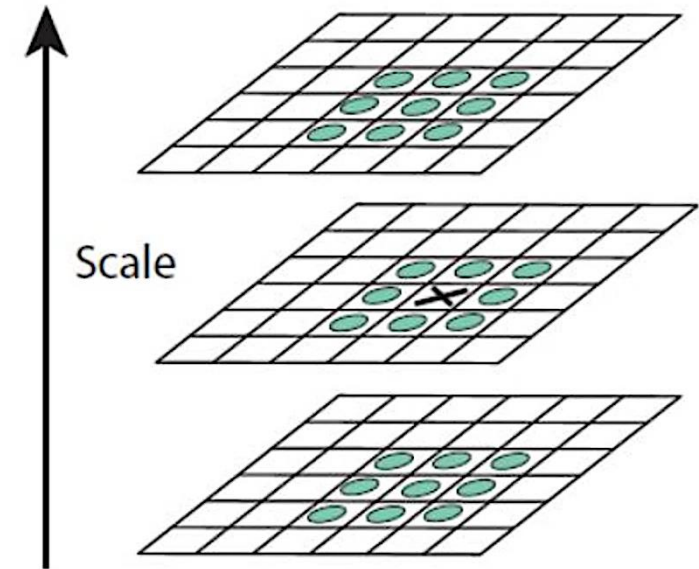
$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

Such a sequence of images convolved with Gaussians of increasing σ_0 constitute a so-called scale space.



Step 1: Candidate Localization

- The keypoints are maxima or minima in the “scale-space-pyramid”.
- Maxima and minima of the difference-of-Gaussian images are detected by comparing a pixel (marked with X) to its 26 neighbors in 3×3 regions at the current and adjacent scales (marked with circles)

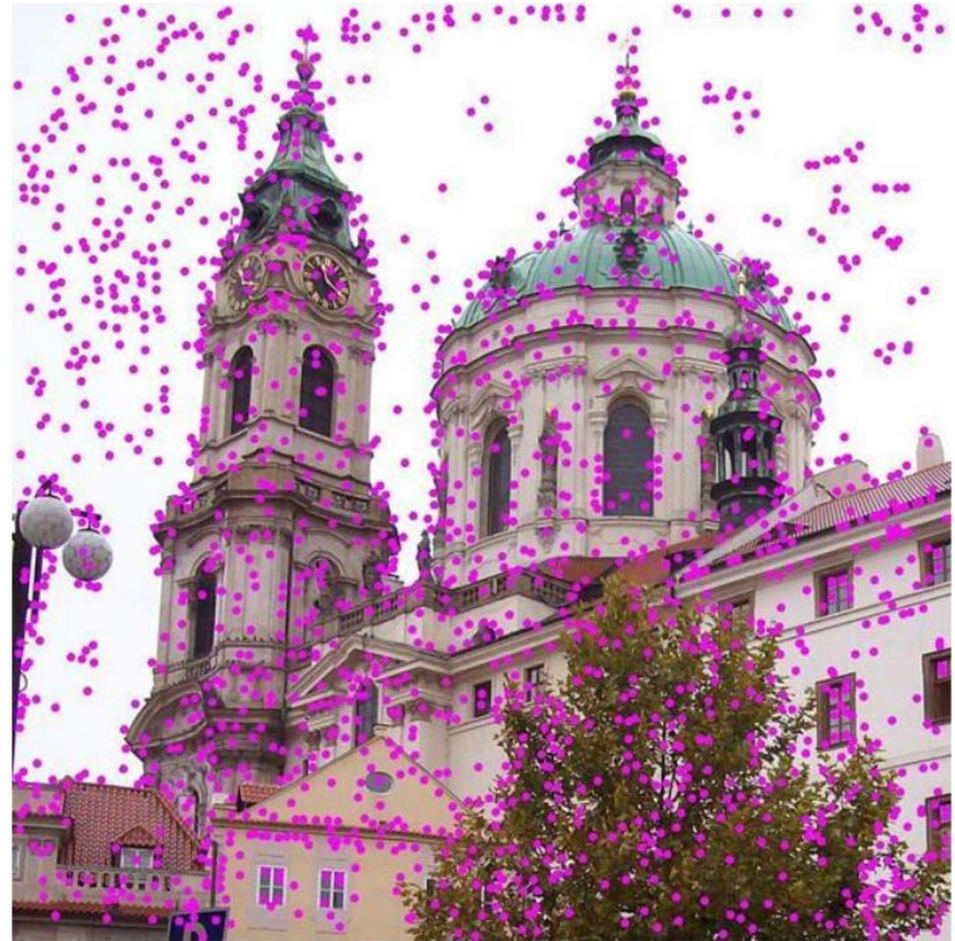


Lowe, D.G. Distinctive Image Features from Scale-Invariant Keypoints. International Journal of Computer Vision 60, 91–110 (2004). <https://doi.org/10.1023/B:VISI.0000029664.99615.94>



Step 1: Candidate Localization

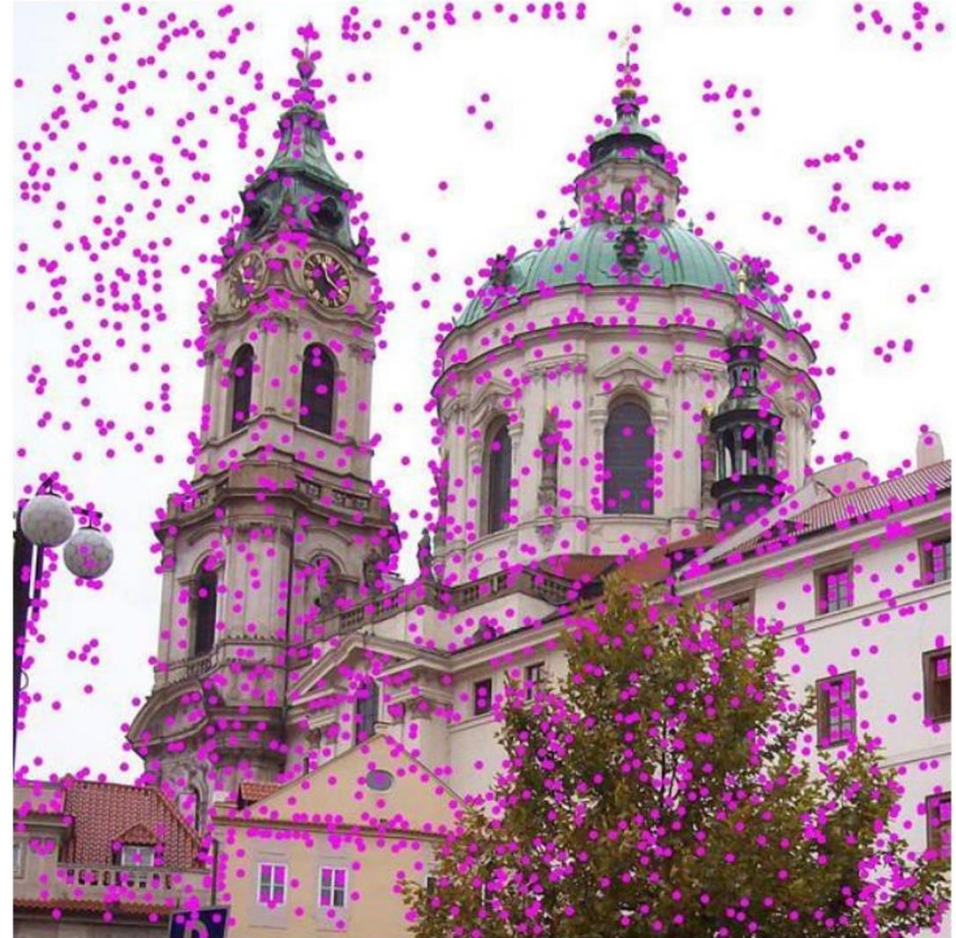
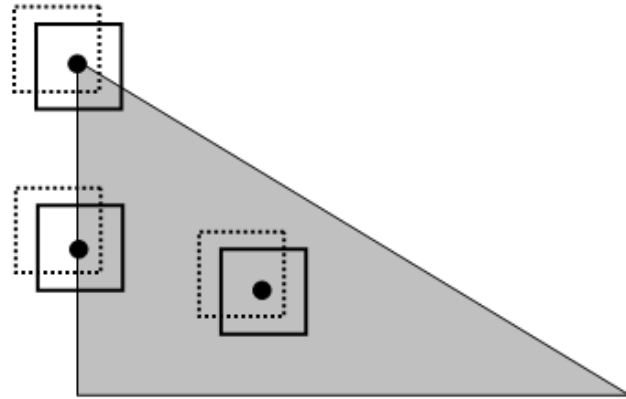
Keypoint candidates





Step 2: Refinement

Idea: keypoint candidates at low-contrast positions or on edges are not stable enough. We have to remove them.



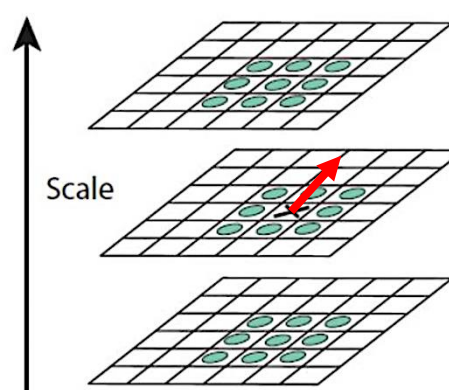


Point-wise calculation is straightforward but inefficient. We need to introduce a little math again



Step 2: Refinement

Let us denote the DoG function in a small 3D neighborhood around a keypoint by a second-order Taylor-series:

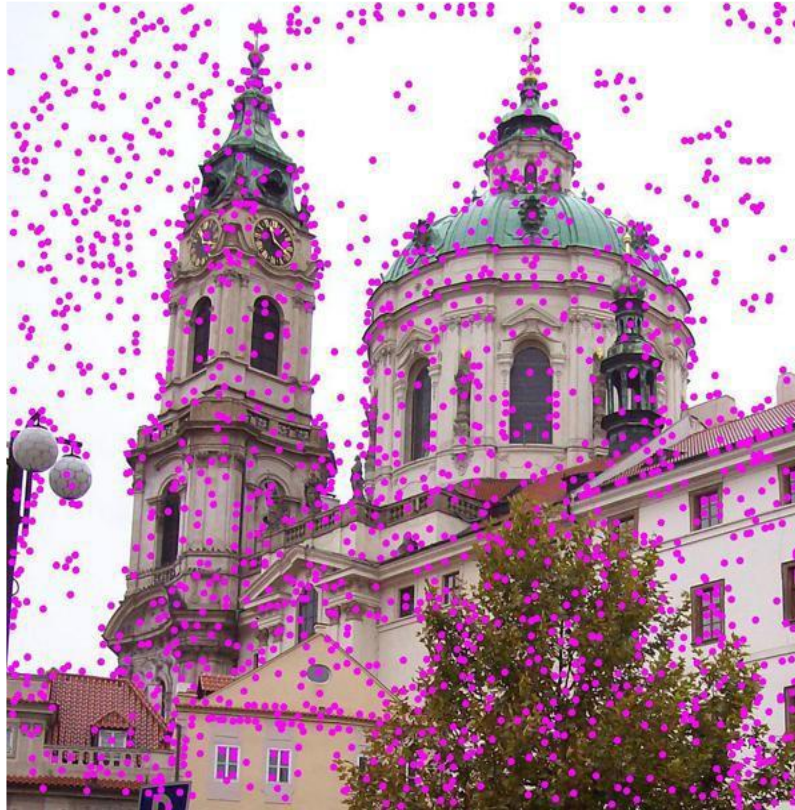

$$\mathbf{x} = (x, y, \sigma)^T \leftarrow \text{Offset to the origin rather than the coordinates}$$
$$D(\mathbf{x}) = D + \underbrace{\frac{\partial D}{\partial \mathbf{x}}^T}_{\text{Gradient Vector}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \underbrace{\frac{\partial^2 D}{\partial \mathbf{x}^2}}_{\text{Hessian Matrix}} \mathbf{x}$$

By setting this to zero, we will have a $\hat{\mathbf{x}}$, at which we can find the **theoretical** local extremum.

We discard current candidate if $\hat{\mathbf{x}} > 0.5$. Why?



Step 2: Refinement



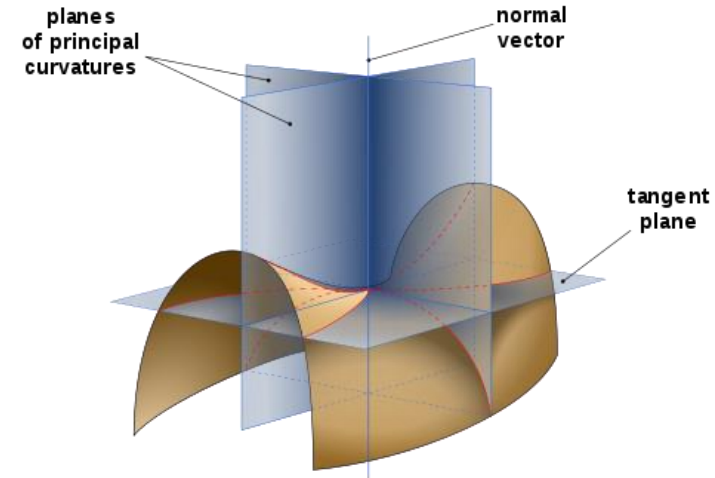
Removal of low-contrast keypoints



Step 2: Refinement

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

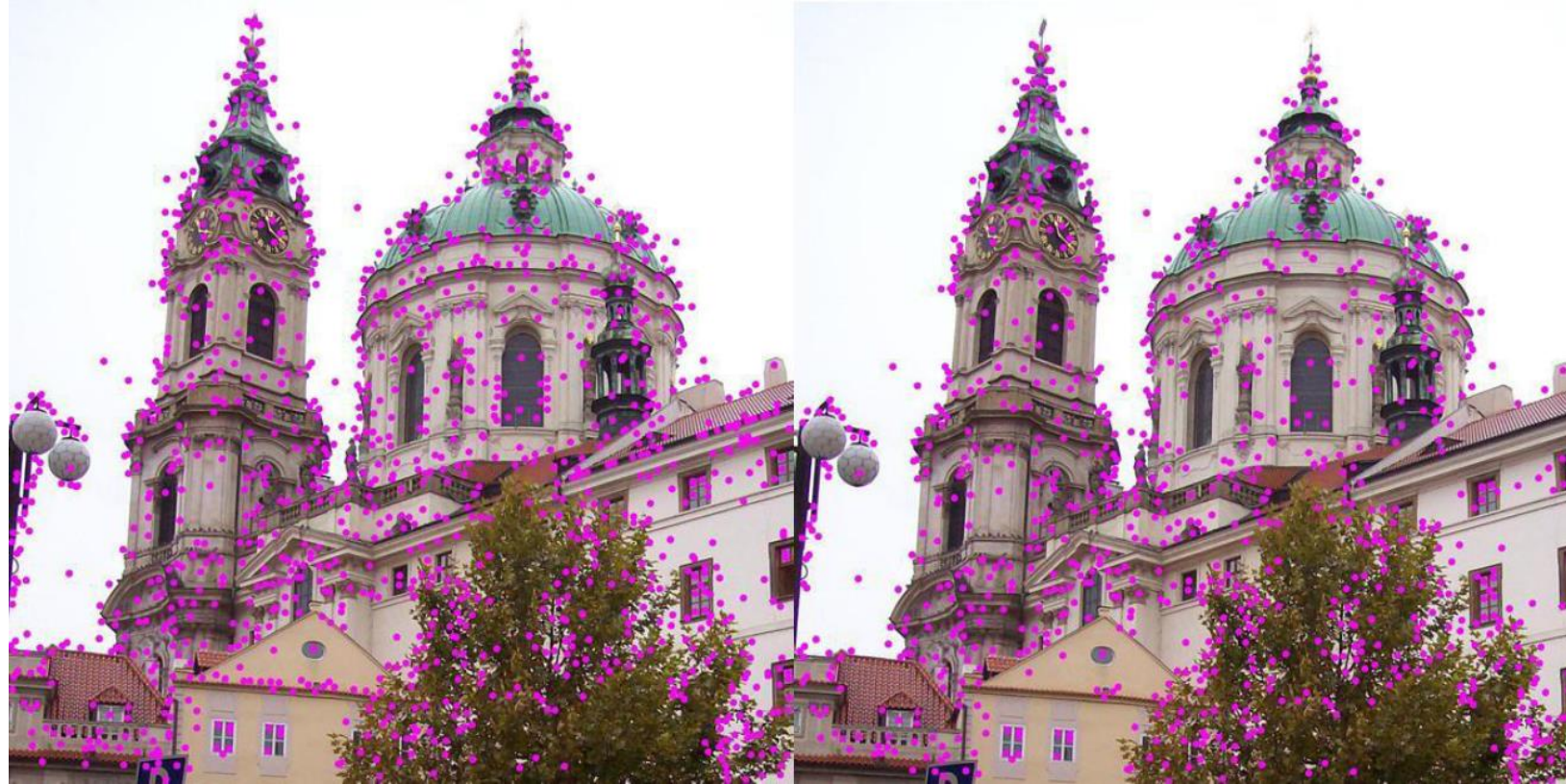
Hessian
Matrix



At an edge candidate, the principal curvature across the edge is much larger than that along the edge. To remove the edge candidates, we calculate **Hessian Matrix**. The eigenvalues of $\mathbf{H}$ are proportional to the principal curvatures of D .



Step 2: Refinement



Removal of high-contrast keypoints residing on edges



Step 3: Orientation Assignment

- Compute the gradient magnitudes and orientations in a small window around the keypoint – at the appropriate scale.

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y).$$

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \tan^{-1}((L(x, y+1) - L(x, y-1)) / (L(x+1, y) - L(x-1, y)))$$

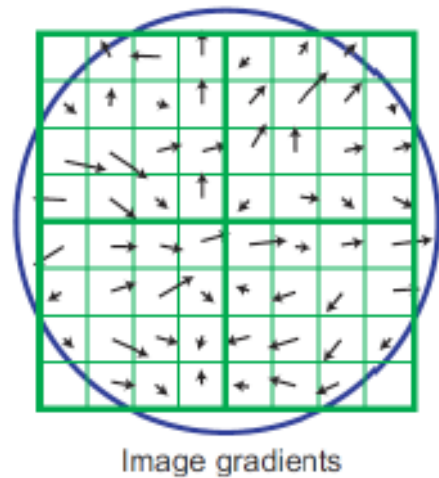
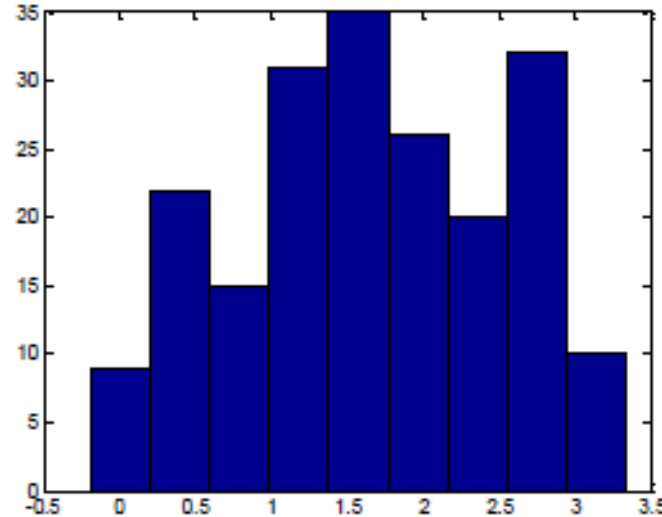


Image gradients

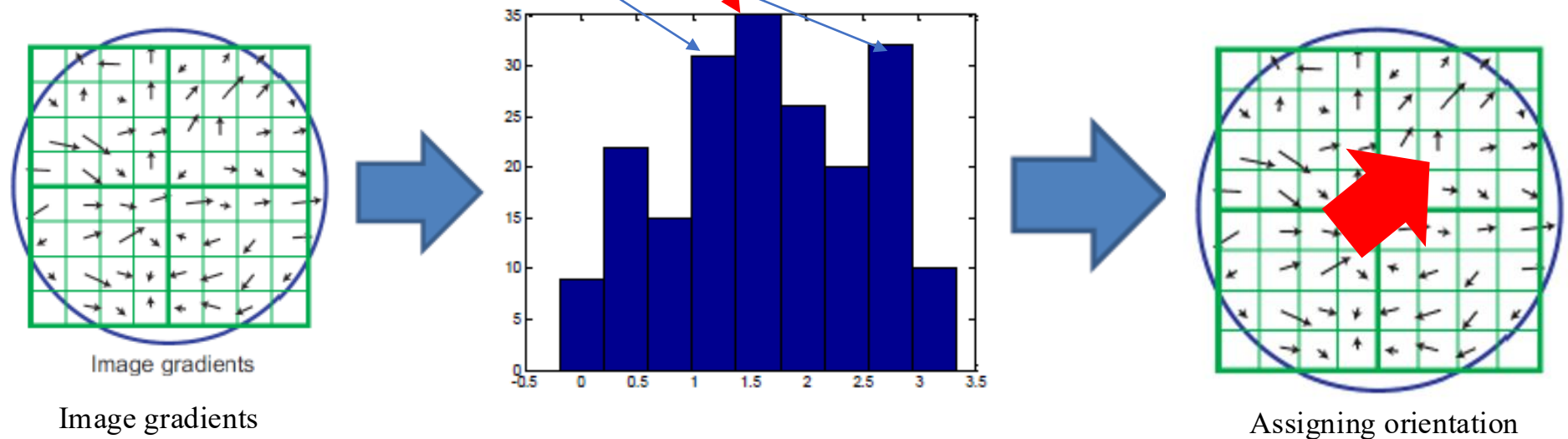


Histogram of gradient orientation – the bin-counts are weighted by gradient magnitudes and a Gaussian weighting function. Usually, 36 bins are chosen for the orientation.



Step 3: Orientation Assignment

- Assign the **dominant orientation** as the orientation of the keypoint.
- In case of multiple peaks or histogram entries more than 0.8 times the peak, create a **separate** descriptor for **each** orientation (they will all have the same scale and location)

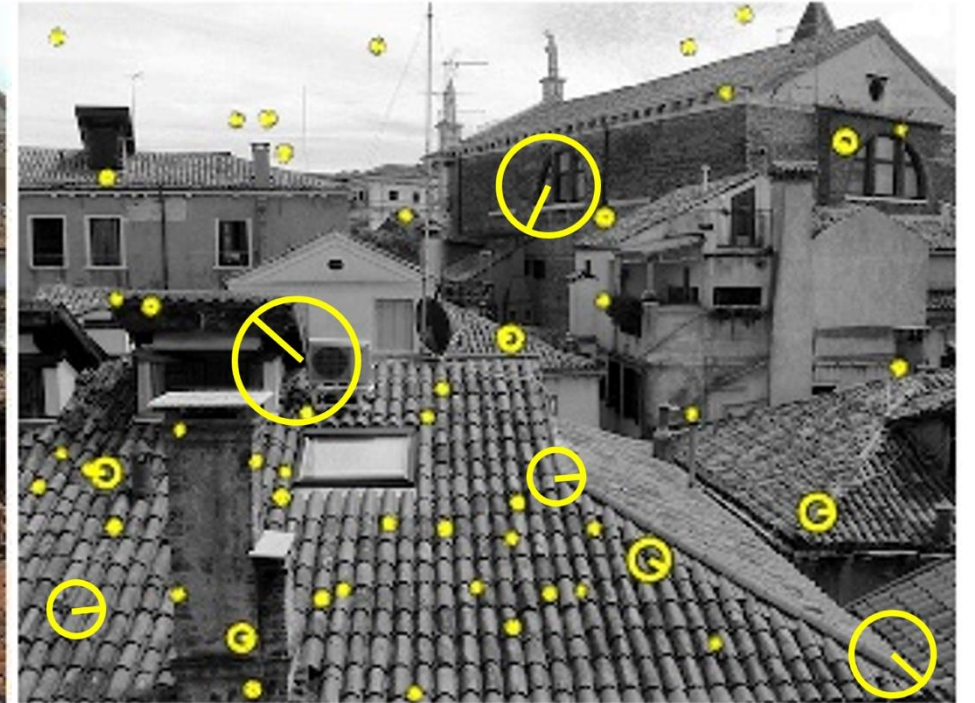




Step 3: Orientation Assignment



Original image

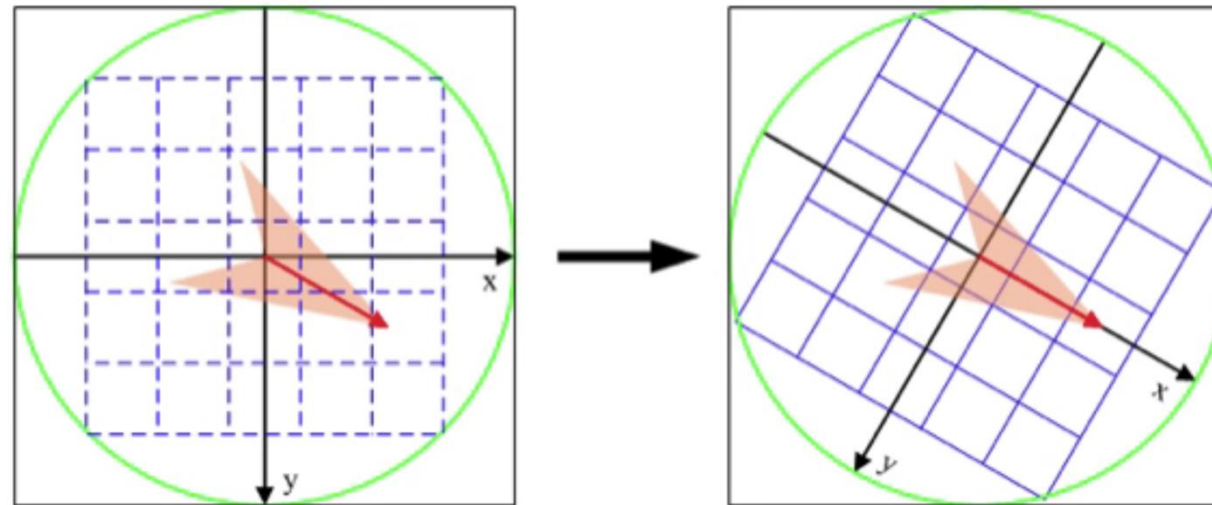


Keypoints and their orientations



Step 4: Build descriptions

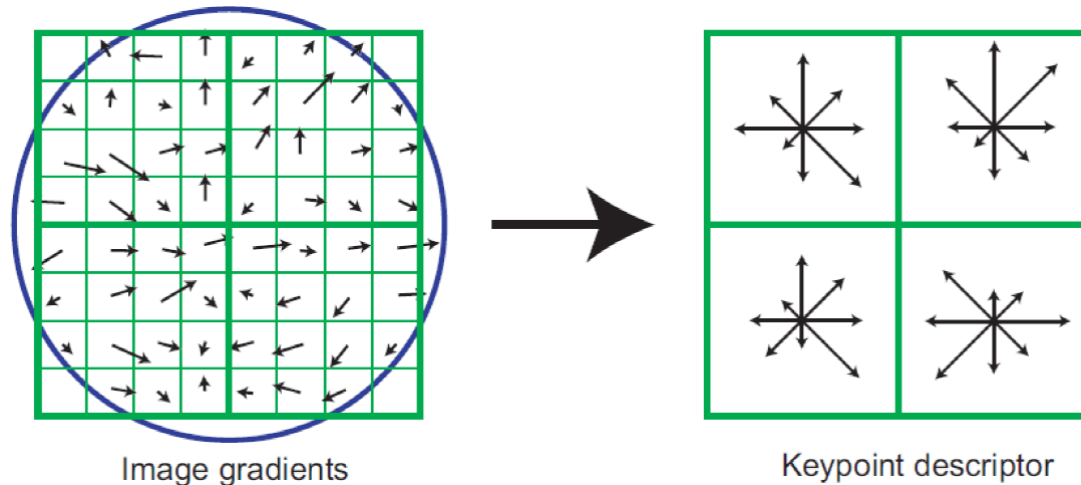
- In order to achieve orientation invariance, the coordinates of the descriptor and the gradient orientations are rotated relative to the keypoint orientation





Step 4: Build descriptions

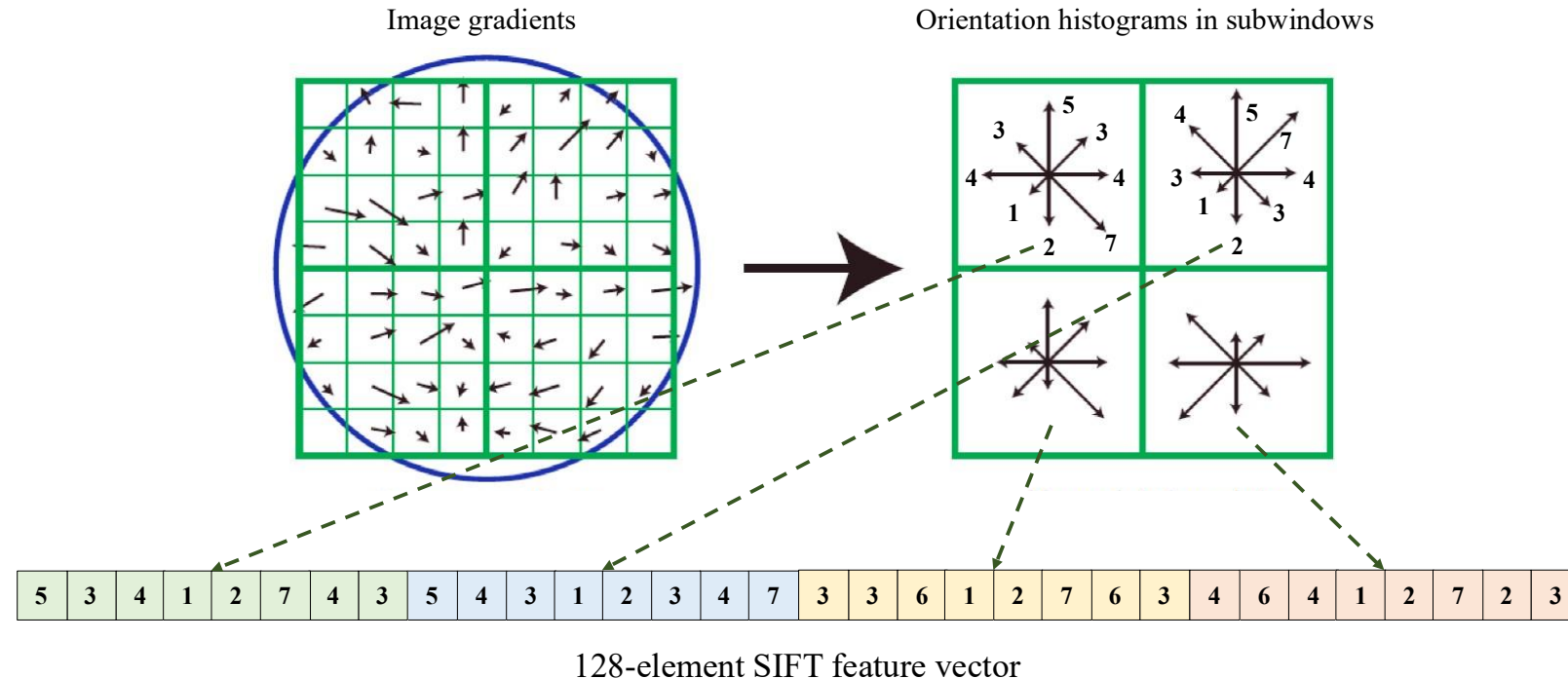
- Consider a small region around the keypoint. Divide it into $n \times n$ cells (for example, $n = 2$, Each cell is of size 4×4).
- Build a **gradient orientation histogram** in each cell. Each histogram entry is weighted by the *gradient magnitude* and a *Gaussian weighting function* with $\sigma = 0.5$ times window width.





Step 4: Build descriptions

- We now have a descriptor of size $rn2$ if there are r bins in the orientation histogram.
- Typical case used in the SIFT paper: $r = 8$, $n = 4$, so length of each descriptor is 128.





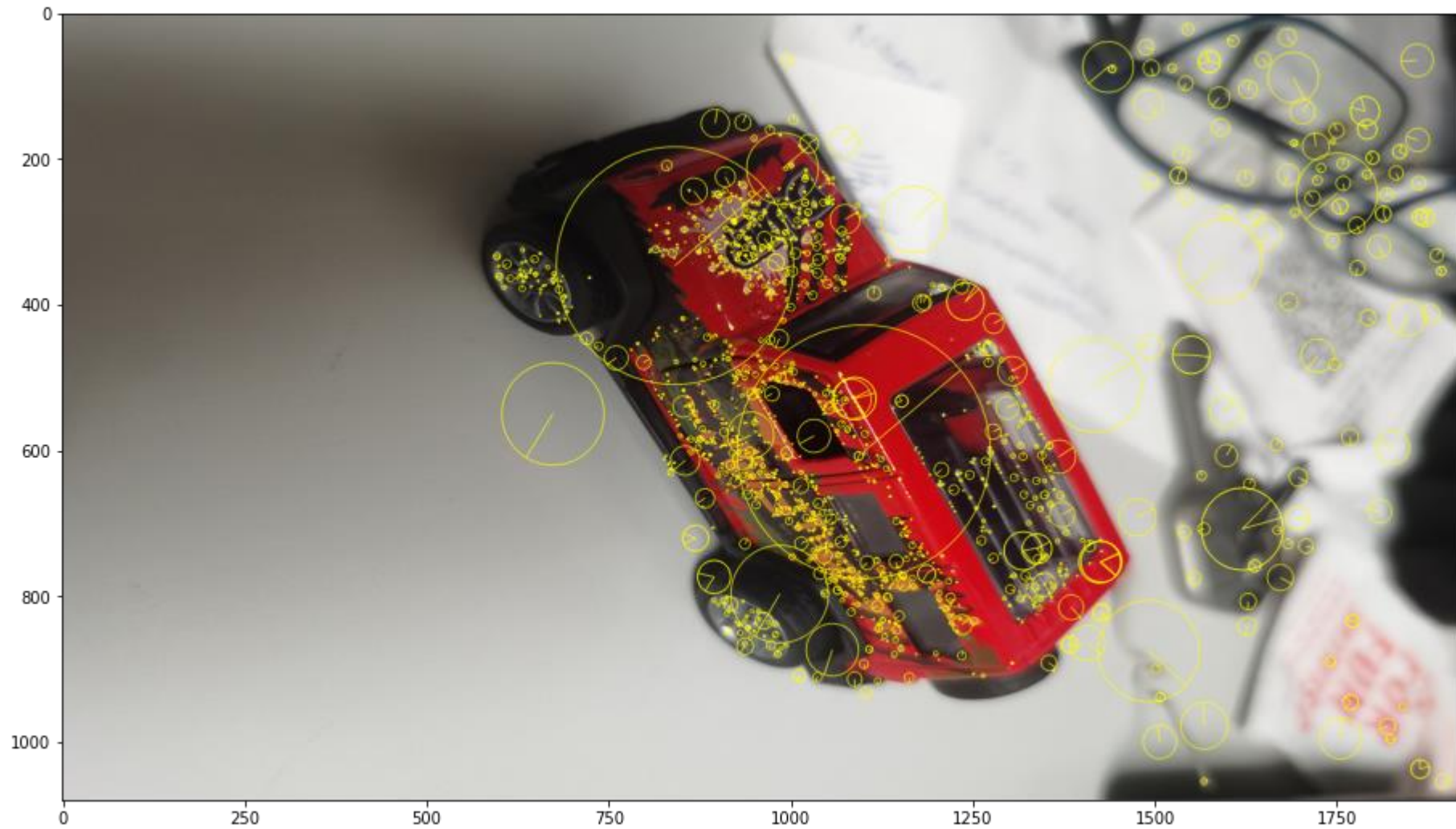
Step 4: Build descriptions

- To achieve scale-invariance, the size of the window should be adjusted as per scale of the keypoint. Larger scale = larger window.



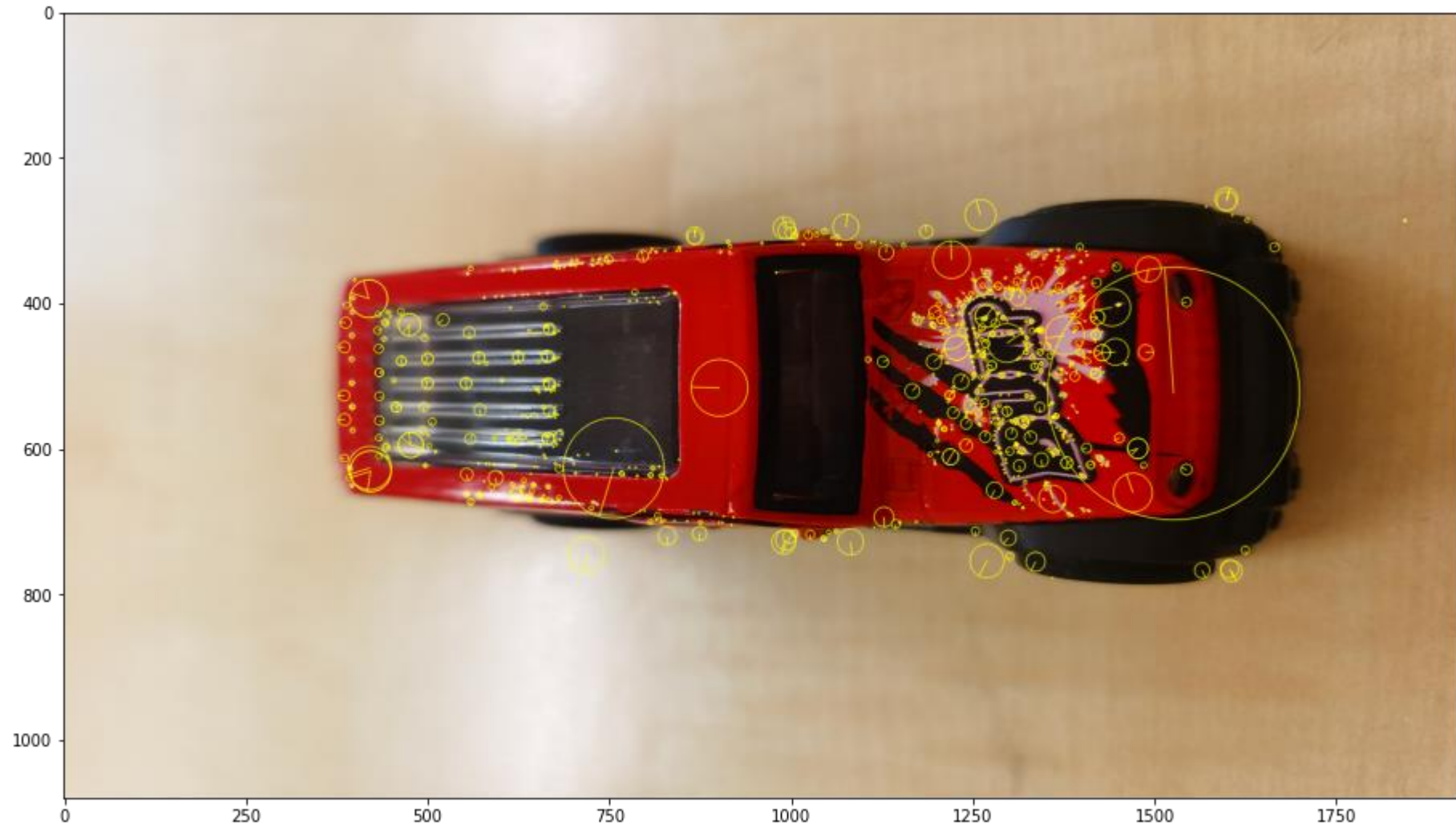


Examples from IMHere



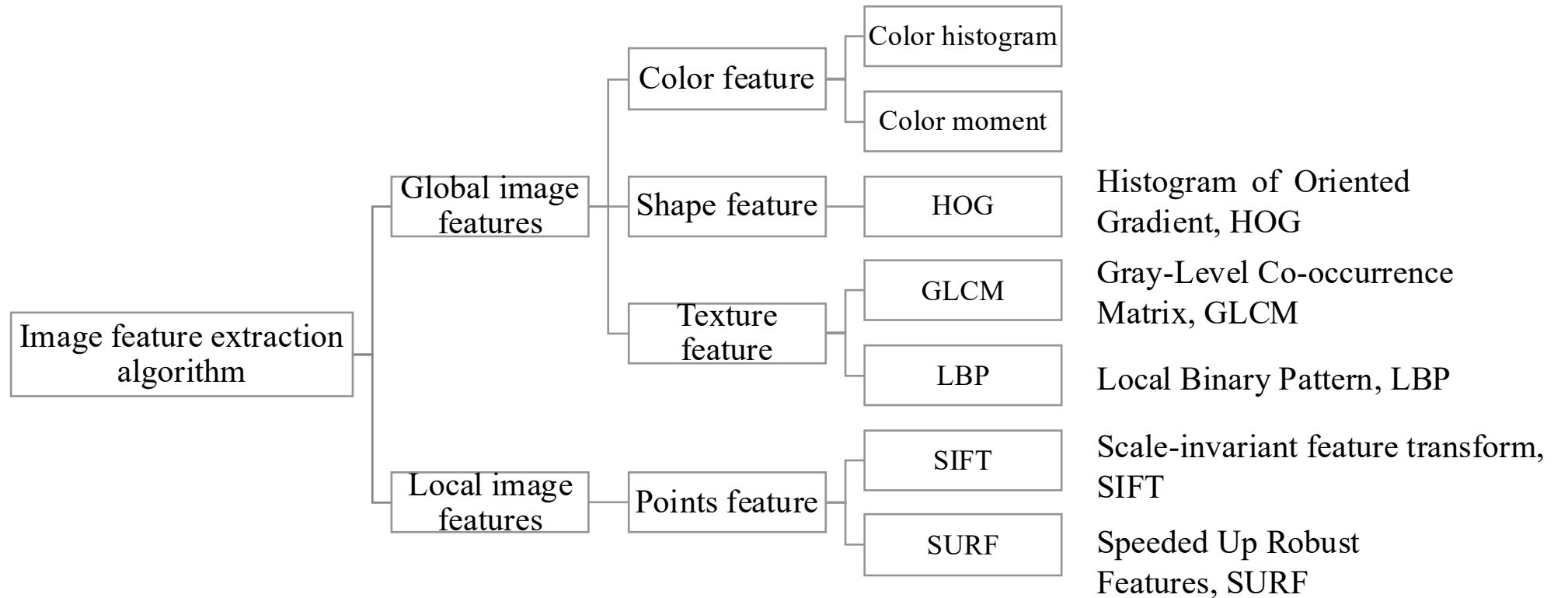


Examples from IMHere



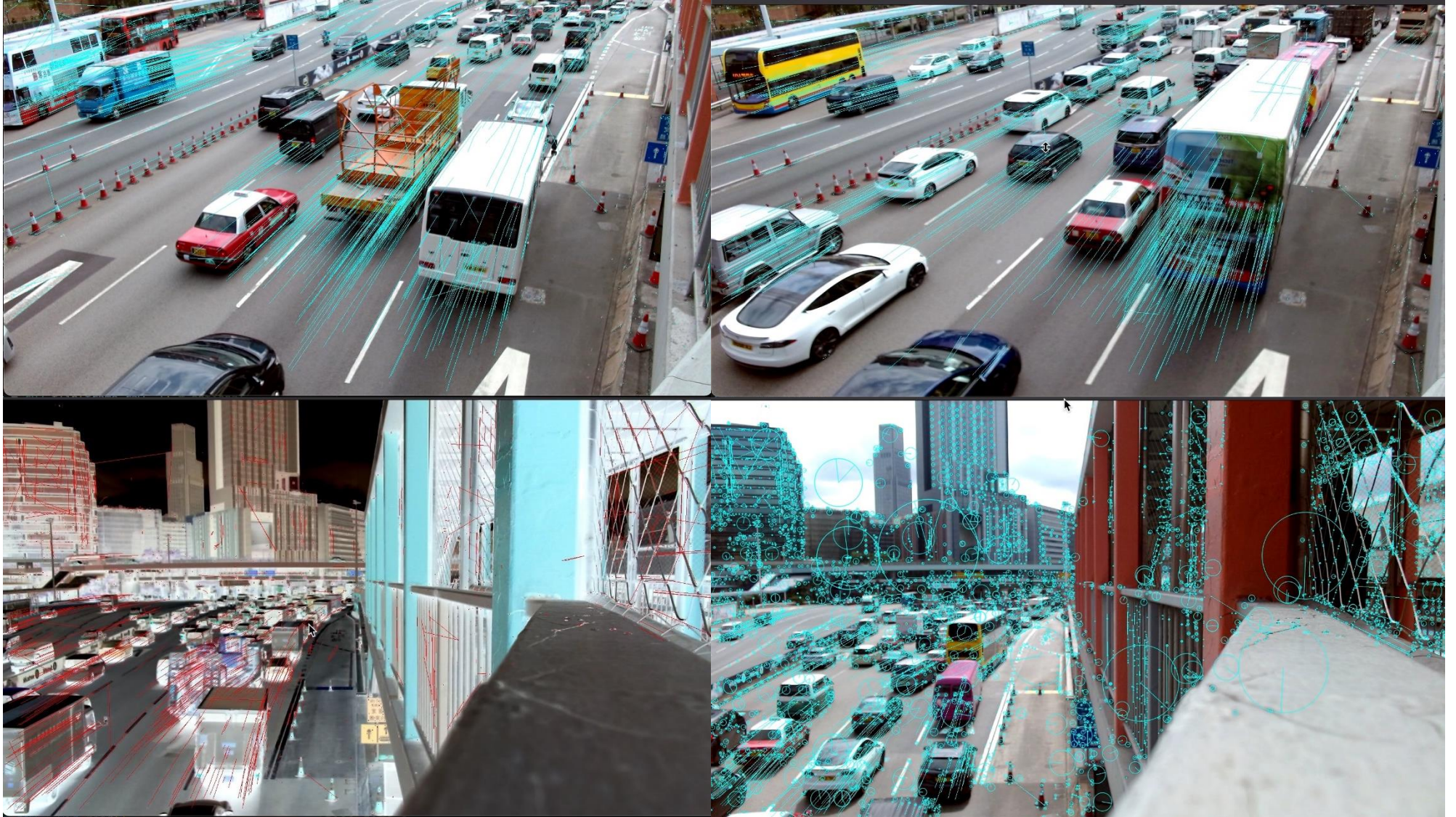


Features





The New Toy





The New Toy

```
1 import cv2,numpy as np,copy
2 # machter reference https://docs.opencv.org/4.x/dc/dc3/tutorial\_py\_matcher.html
3
4 # declare a video capture object
5 vid = cv2.VideoCapture(0)
6 inversion_on=False
7 keypoints_on=False
8
9 sift = cv2.SIFT_create()
10 detector=sift
11
12 bf_matcher=cv2.BFMatcher_create()
13 FLANN_INDEX_KDTREE = 1
14 index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
15 search_params = dict(checks=100)
16 flann = cv2.FlannBasedMatcher(index_params, search_params)
17 matcher=flann
18 kp_old,des_old=None,None
19 traces={}
20
21 def distance(p1, p2):
22     dis = ((p2[0] - p1[0]) ** 2 + (p2[1] - p1[1]) ** 2) ** 0.5
23     return dis
24
```



The New Toy

```
25 while True:
26     # capture the video frame by frame
27     ret, frame_raw = vid.read()
28     frame = cv2.flip(frame_raw, 1) # optional
29     # get dimensions of the frame
30     h, w, c = frame.shape
31     print(h,w,c)
32
33     #
34     gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
35     kp, des = detector.detectAndCompute(gray, None)
36     #print(type(kp))
37
38     if kp_old!=None:
39         matches = matcher.knnMatch(des,des_old, k=2)
40         good = []
41         for m,n in matches:
42             if m.distance < 0.8*n.distance:
43                 good.append(m)
44         #matches=bf_matcher.match(des,des_old)
45
46         new_traces={}
47         for match in good:
48             pt1,pt2=kp[match.queryIdx].pt,kp_old[match.trainIdx].pt
49             pt1,pt2=[int(pt1[0]),int(pt1[1])],[int(pt2[0]),int(pt2[1])]
50             if distance(pt1,pt2)>200: continue
```



The New Toy

```
52         if (match.trainIdx in traces):
53             len_pts=len(traces[match.trainIdx])
54             new_traces[match.queryIdx]=[pt1]+copy.deepcopy(traces[match.trainIdx][:20 if len
55             pts=np.array(new_traces[match.queryIdx],dtype='int32')
56             #print(pts)
57             cv2.polylines(frame,[pts],False,(255, 255, 0),1)
58         else:
59             cv2.line(frame,pt1,pt2,(255, 255, 0),1)
60             new_traces[match.queryIdx]=[pt1,pt2]
61         traces=new_traces
62
63     kp_old,des_old=kp,des
64     if keypoints_on: cv2.drawKeypoints(frame, kp, frame, (255, 255, 0), cv2.DRAW_MATCHES_FLAGS_DI
65
66     # display the resulting frame
67     cv2.imshow('COMP 4423', 255-frame if inversion_on else frame)
68
69     key=cv2.waitKey(1) & 0xFF
70     # quit when 'q' is pressed
71     if key== ord('q'):
72         break
73     if key==ord('i'):
74         inversion_on=not inversion_on
75     if key==ord('k'):
76         keypoints_on=not keypoints_on
77
78     # release the cap object
```



TODO: Study following concepts by yourself

1. Mahalanobis Distance
2. Color Moment
3. HOG
4. SURF
5. The New Toy source code



After-class Reflection and Beyond

1. What is quantization, and why is it useful in image feature extraction?

Give an example of how a human visual concept can be converted into numerical values.

2. Simple if-else rules may work for clean examples, but often fail for real images. Why?

Discuss how illumination, viewpoint, shadows, or noise can make feature values uncertain.

3. What is the difference between Euclidean distance and cosine similarity?

In what kind of situation might cosine similarity be more appropriate than Euclidean distance?

4. Mahalanobis distance considers the distribution of data. Why might this be useful?

Explain using an example where data are stretched or elongated in one direction in feature space.

5. What is the difference between global image features and local image features?

Give one example of each and describe what kind of information it captures.

6. What information does a color histogram capture, and what information does it ignore?

For example, can two very different images have similar color histograms?

7. How does the choice of histogram bin size affect a color histogram feature?

What might happen if the bins are too large or too small?

8. How does Local Binary Pattern, or LBP, describe local texture?

Explain the basic idea of comparing neighboring pixels with the center pixel.

9. What makes SIFT useful for matching images taken from different viewpoints or scales?

Discuss how scale invariance, orientation assignment, and local descriptors help SIFT match corresponding points.



After-class Reflection and Beyond

9. What makes SIFT useful for matching images taken from different viewpoints or scales?

Discuss how scale invariance, orientation assignment, and local descriptors help SIFT match corresponding points.

10. SIFT first detects many candidate keypoints and then removes unstable ones. Why is this two-stage strategy useful?

11. How does Gaussian blur help SIFT build a scale space?

Explain the connection between blurring an image and observing an object at different scales.



Thank You!